



KodeKloud

© Copyright KodeKloud

Follow us on <https://kodekloud.com/> to learn more.

EC2 Instance and IAM Role

© Copyright KodeKloud

- Introduction to why do we need IAM role
- How does it improve security , consideration while designing the application with other AWS service
- How to assume role
- What happens when we assume role in the background

Material

An IAM role is an IAM identity that you can create in your account that has specific permissions. An IAM role is similar to an IAM user in that it is an AWS identity with permissions policies that determine what the identity can and cannot do in AWS. However, instead of being uniquely associated with one person, a role is intended to be assumable by anyone who needs it. Also, a role does not have standard long-term credentials such as a password or access keys associated with it. Instead, when you assume a role, it provides you with temporary security credentials for your role session. For more information on how to create IAM roles and grant permissions with them, see [IAM roles for Amazon EC2](#).

Applications must sign their API requests with AWS credentials. Therefore, if you are an application developer, you need a strategy for managing credentials for your applications that run on EC2 instances. For example, you can securely distribute your AWS credentials to the instances, enabling the applications on those instances to use your credentials to sign requests, while protecting your credentials from other users. However, it's challenging to securely distribute credentials to each instance, especially those that AWS creates on your behalf, such as Spot Instances or instances in Auto Scaling groups. You must also be able to update the credentials on each instance when you rotate your AWS credentials.

1. Create an IAM role.

We designed IAM roles so that your applications can securely make API requests from your instances, without requiring you to manage the security credentials that the applications use. Instead of creating and distributing your AWS credentials, you can delegate permission to make API requests using IAM roles as follows:

2. Define which accounts or AWS services can assume the role.

3. Define which API actions and resources the application can use after assuming the role.

4. Specify the role when you launch your instance, or attach the role to an existing instance.

5. Have the application retrieve a set of temporary credentials and use them.

Instance Metadata

Instance metadata is data about your instance that you can use to configure or manage the running instance. Instance metadata is divided into categories, for example, host name, events, and security groups. You can also use instance metadata to access *user data* that you specified when launching your instance. For example, you can specify parameters for configuring your instance, or include a simple script.

Instance metadata is available on IP address 169.254.169.254 , from where you can fetch all the information related to the ec2 instance.

Instance profiles

What is instance profile and the use of it.

Retrieve security credentials from instance metadata

```
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/s3access
```

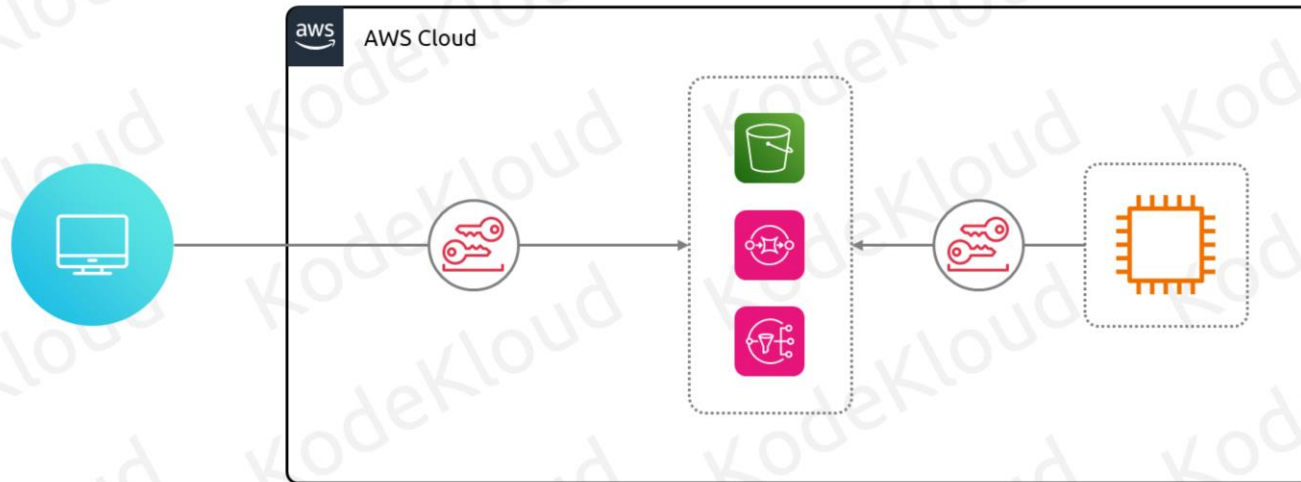
```
{ "Code" : "Success", "LastUpdated" : "2012-04-26T16:39:16Z", "Type" : "AWS-HMAC", "AccessKeyId" :  
"ASIAIOSFODNN7EXAMPLE", "SecretAccessKey" : "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY",  
"Token" : "token", "Expiration" : "2017-05-17T15:09:54Z" }
```

AWS CLI command which would create connect to S3 automatically without needing to fetch any credentials it is taken care automatically.

Actions on IAM ROLE

1. It can be attached to a running instance running and stopped
2. Replace IAM role running only
3. Detach IAM role running and stopped

Need for an IAM Role



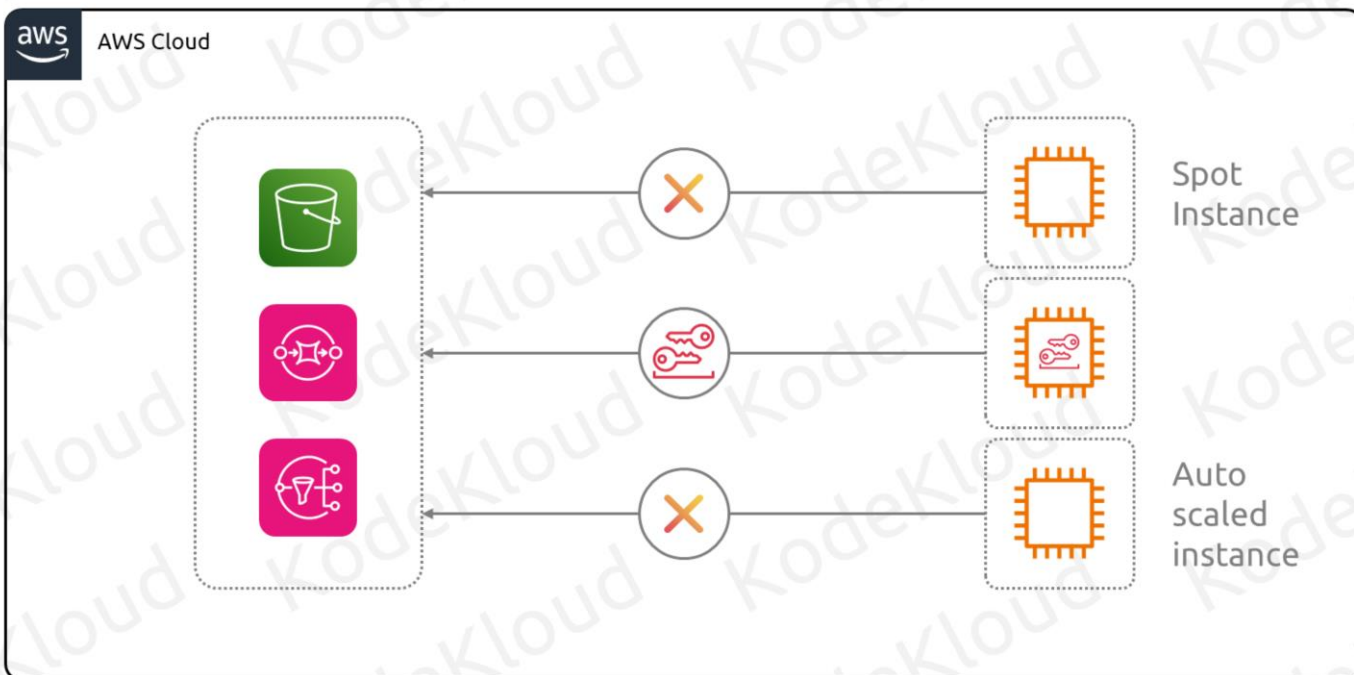
© Copyright KodeKloud

Why Do we need IAM Role

Let's try to understand the need for IAM role.

Application running on EC2 instance or on a laptop, if it has to interact with AWS service.

Applications must sign their API requests with AWS credentials, Therefore, if you are an application developer, you need a strategy for managing credentials for your applications that run on EC2 instances.



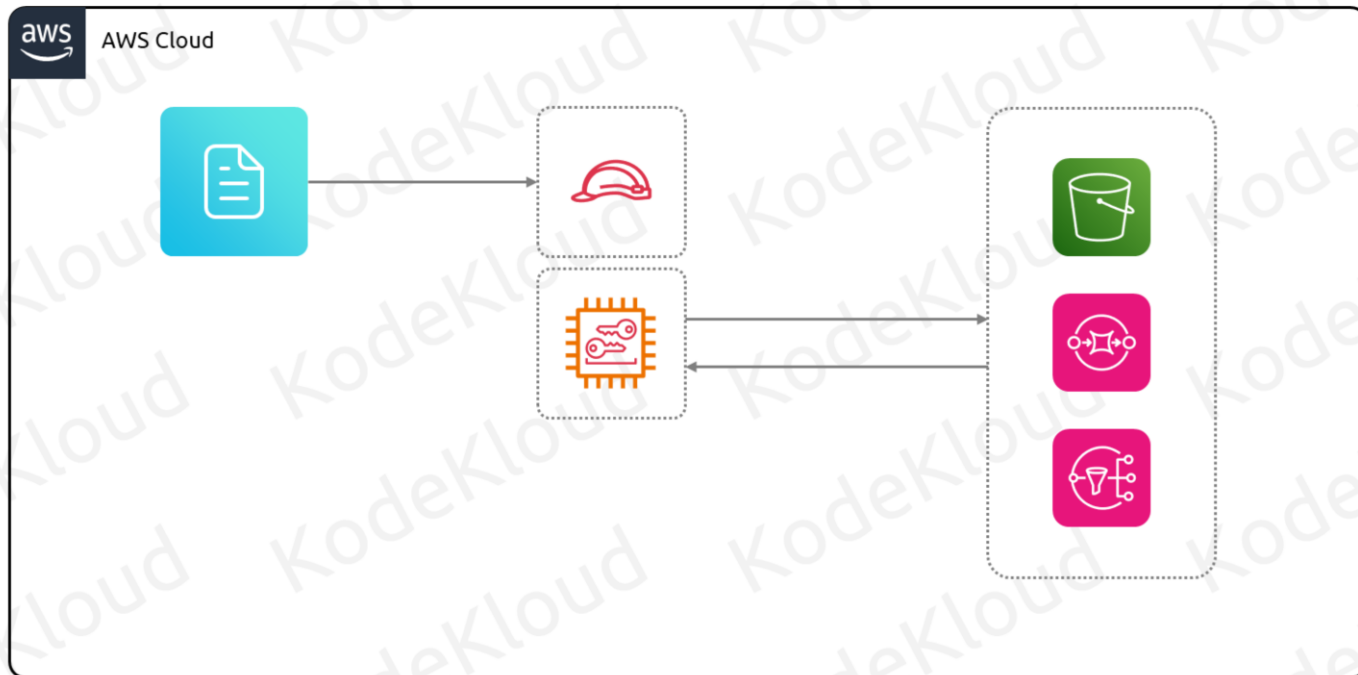
© Copyright KodeKloud

One option is to use access key and secret key, this works well from a single EC2 which does not get terminated.

However, it's challenging to securely distribute credentials to new instance, especially those that AWS creates on your behalf, such as Spot Instances or instances in Auto Scaling groups. You must also be able to update the credentials on each instance when you rotate your AWS credentials.

As you can see, if credentials are not present on the instance the request to AWS services are rejected and static credentials is not a scalable approach.

Working of IAM Role



© Copyright KodeKloud

This is where IAM role comes into picture

An IAM role is an IAM identity that you can create in your account that has specific permissions.

An IAM role is similar to an IAM user, it is an AWS identity with permissions policies that determine what the identity can and cannot do in AWS. However, instead of being uniquely associated with one person, a role is intended to be

assumable by anyone who needs it. Also, a role does not have standard long-term credentials such as a password or access keys associated with it. Instead, when you assume a role, it provides you with temporary security credentials for your role session.

IAM policy with required permission is attached to the role and then this IAM role is associated to the EC2 instance while creating or to a running instance.

Temporary credentials are fetched and based on the allowed permission the application running on the EC2 instance can communicate with respective services.

Now let's look at what is happening under the hood of IAM role.

EC2 Metadata

```
Instance metadata
```

```
$ curl http://169.254.169.254/latest/meta-data/  
...  
...  
...  
Iam/  
instance-action/  
instance-id/  
  
...  
... *  
  
...  
  
* *  
  
* *
```



© Copyright KodeKloud

Before we get into IAM role, few basic to be understood, EC2 *Instance metadata* is data about your instance that you can use to configure or manage the running instance. Metadata is unique to the respective EC2 instance and can only be accessed on IP 162.254.169.254.

EC2 metadata has many categories, which can be listed by performing a curl from your respective ec2 instance only.

From the output, IAM, instance-action & instance-id are few of the categories

EC2 Metadata

```
$ curl http://169.254.169.254/latest/meta-data/iam/security-credentials/s3access
{
  "Code" : "Success",
  "LastUpdated" : "2012-04-26T16:39:16Z",
  "Type" : "AWS-HMAC",
  "AccessKeyId" : "ASIAIOSFODNN7EXAMPLE",
  "SecretAccessKey" : "wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY",
  "Token" : "IQoJb3JpZ2luX2VjEB0aDmFwLXNvdX..... . . ,
  "Expiration" : "2023-07-31T15:09:54Z"
}
```

© Copyright KodeKloud

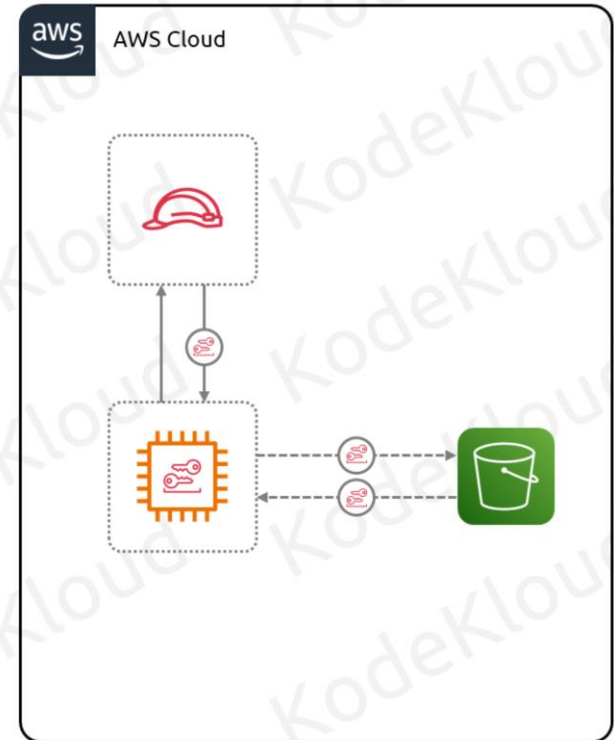
S3 access is the IAM role attached to the EC2 instance

When we perform curl with IAM role name which is attached to the ec2 instance, the response from STS service is temporary credentials which can be used only from the EC2 instance and it is short lived.

The temporary credentials would look like this with accesskey, secretaccesskey and token.

Working of IAM Role

```
$ aws s3 ls s3://example-bucket  
  
File1.txt  
file2.txt
```



© Copyright KodeKloud

In a ec2 instance which has an IAM role attached to it, Using AWS CLI, when we execute a command to list content of a s3 bucket.

In the background

1. CLI would fetch temporary credentials
2. Sign the requests to AWS s3 service and post it to the respect service API endpoint and retrieve the list of S3 bucket

contents.

EC2 Instance and IAM Role



© Copyright KodeKloud

Going back to our previous problem statement, when EC2 instance scales up,

1. IAM role is also attached the new instance
2. Application using aws sdk can fetch the temporary credentials seamlessly and interact with required aws service based on the operation which are permitted in the policy.

EC2 IAM Role – Summary

IAM role
credentials are
temporary

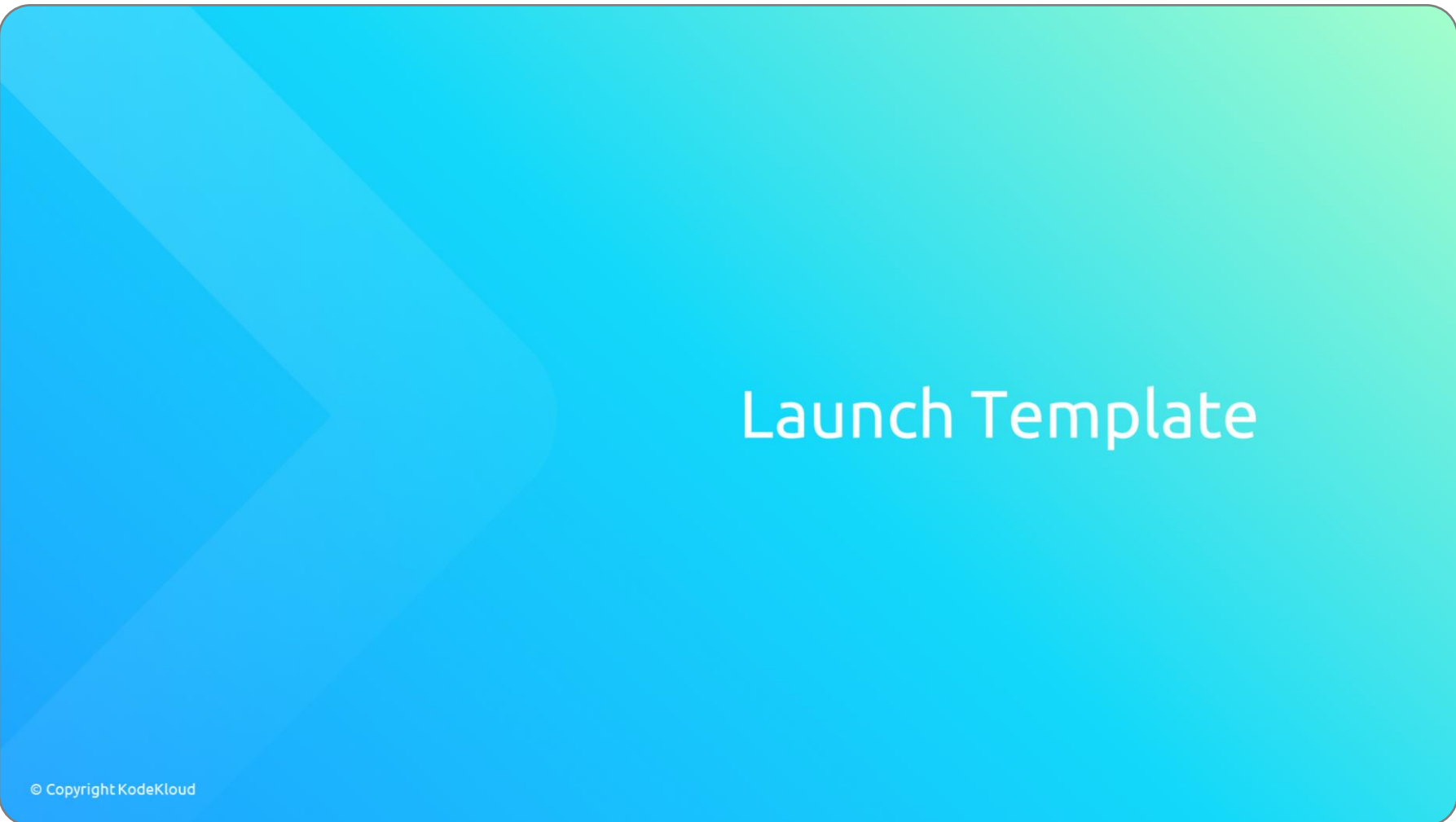
Works only from
EC2 instance

EC2 instance
attached to
single role

Role can be
attached to
multiple EC2

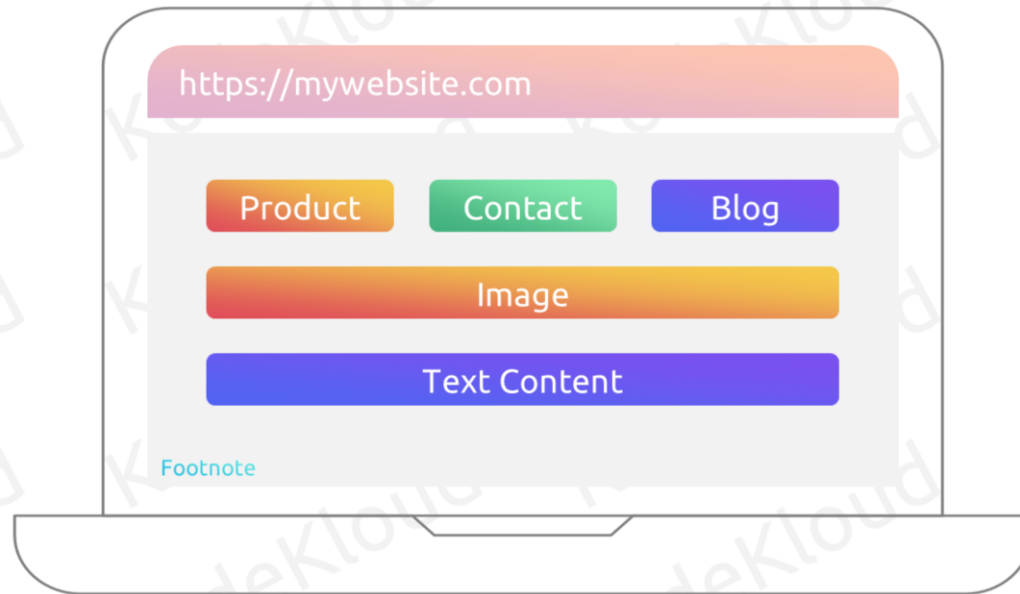
AWS SDKs, AWS CLI, and Tools for Windows PowerShell fetch credentials automatically from EC2 instance metadata service

Why Do we need AIM Role



Launch Template

Why Do We Need Launch Template?



© Copyright KodeKloud

Imagine you are building a website and after building a few of them you identify a pattern, every website would have a home page, contact page, product page and blog, on your home screen you have three sections with welcome image, text content and foot note.

To ease your work for the next website it is practical to build a template which has all the common features, which can be used readily instead of building from scratch.

Advantages of Template



Standardization



Scalability

© Copyright KodeKloud

The main advantage of templates are ,

1. standardization and
2. increase in scale, with the same template you can build multiple websites with less efforts.

Launch Template – EC2 Common Features



IAM Role



Security Group



Network Configuration



Instance Type



AMI

Many more...

Similarly, when you create a EC2 instance you can find common elements like

1. IAM Role
2. Security group
3. Network configuration
4. Instance type

5. AMI

Instead of providing these common parameters each and every time you create an instance, better approach would be to use a launch template, which provides standardization, repeatability and control on type of EC2 instance which is running in your account.

Features of Launch Template

01

Instance Config

1. AMI
2. EBS volume
3. Security group
4. IAM profile
5. Network settings
6. Instance tag

02

Versioning and Update

03

Customization

© Copyright KodeKloud

Features of Launch Template

1. **Comprehensive Instance Configuration:** Launch Templates allow you to define a wide range of instance configuration settings, including instance type, Amazon Machine Image (AMI), block device mappings, security groups, IAM instance profile, network settings, instance tags, and more.

2. **Versioning and Updates:** Launch Templates support versioning, which means you can create multiple versions of a template. This facilitates easy updates and changes over time without recreating the entire template. You can update an existing launch template without disrupting your existing instances.
3. **Parameterization and Customization:** Launch Templates support parameterization, allowing you to create a base template with common settings and then customize or override specific parameters when launching instances. This helps you create instances with slight variations from the base template.

Features of Launch Template

04

Instance Type and Purchase Options

05

Tagging and Metadata

06

API Support

07

Instance Protection

08

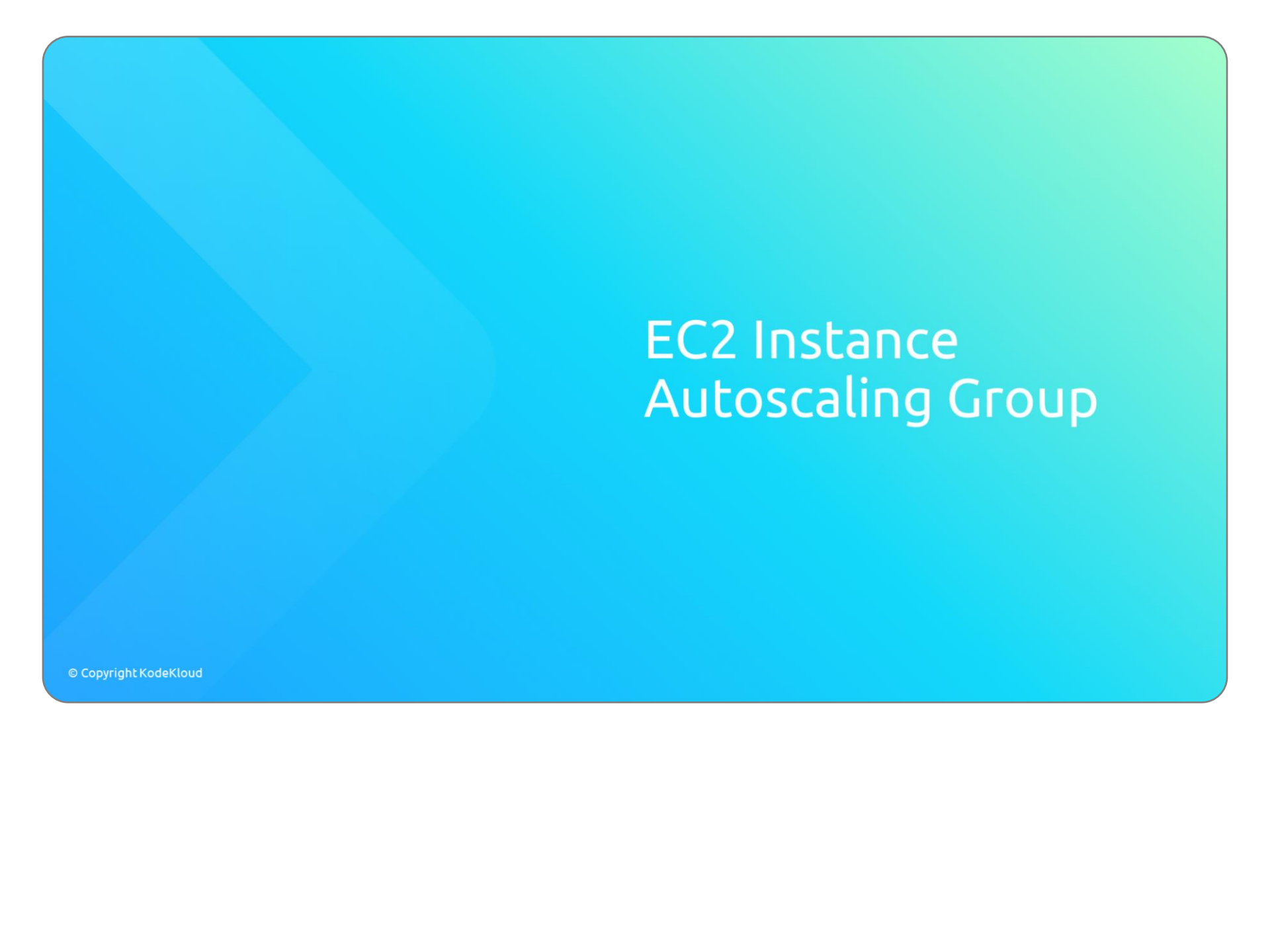
User Data

© Copyright KodeKloud

1. **Instance Types and Purchasing Options:** You can specify different instance types and purchasing options (On-Demand, Reserved Instances, Spot Instances) within the same launch template. This flexibility allows you to optimize cost and performance based on your workload requirements.
2. **Tagging and Metadata:** You can include tags and metadata in your launch templates, making it easier to organize, manage, and categorize instances for billing, monitoring, and organizational purposes.
3. **API and Automation Support:** Launch Templates can be managed using the AWS Management Console, AWS

Command Line Interface (CLI), AWS SDKs, and other automation tools. This allows you to integrate instance provisioning into your development and deployment workflows.

4. **Flexible AMI Selection:** Launch Templates allow you to specify different AMIs for different instances, enabling you to launch instances based on various operating systems or software configurations.
5. **Instance Termination Protection:** You can enable instance termination protection at the launch template level to prevent accidental termination of instances.
6. **User Data and Instance Initialization:** Launch Templates support user data, which allows you to pass scripts and data to instances at launch time. This can be useful for automating instance initialization tasks.



EC2 Instance Autoscaling Group

Autoscaling



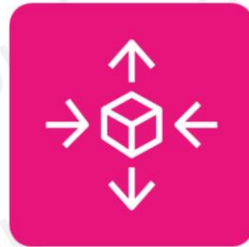
© Copyright KodeKloud

Imagine a bakery which can make cup cakes based on the requirement, more customers walk into the store asking for cup cakes more cup cakes are baked by turning on, more ovens. When the current oven reaches 80% capacity, the next oven is activated to ensure baking continues uninterrupted. Similarly, when the oven is only utilising 20% of its capacity, it is shut off after baking to conserve resources.



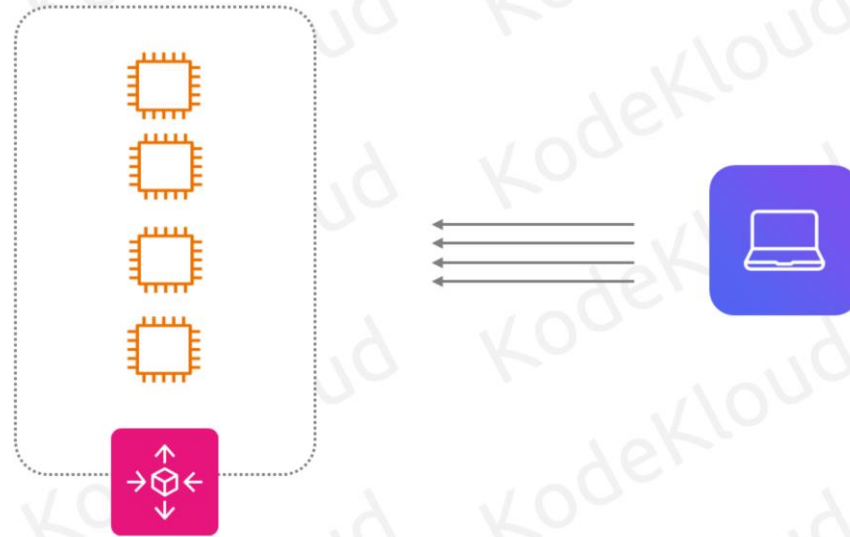
Here are going to show what happens when the capacity reduces.

What Is EC2 Autoscaling?



AWS Autoscaling

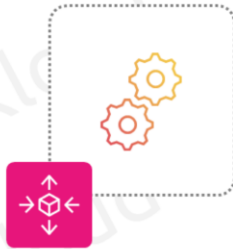
Similarly in AWS Auto scaling group is used to dynamically scale EC2 instance based on the demand. As the number of request comes increases, to meet the demand new EC2 instances are also provisioned.



© Copyright KodeKloud

Similarly in AWS Auto scaling group is used to dynamically scale EC2 instance based on the demand. As the number of request comes increases, to meet the demand new EC2 instances are also provisioned.

Features of Autoscaling



01 Scaling Policy

- Manual
- Dynamic
- Scheduled

02 Auto Healing

© Copyright KodeKloud

Let's look at the features of auto scaling group and how it is used:

When the scaling policy is in effect, the Auto Scaling group adjusts the desired capacity of the group, between the minimum and maximum values that you specify, and launches or terminates the instances as needed.

1. **Scaling Policy** : This is the main feature of ASG base do on which the instances are increased and decreased. Broadly

scaling policy can be classified into

1. Manual
2. Dynamic
3. Scheduled

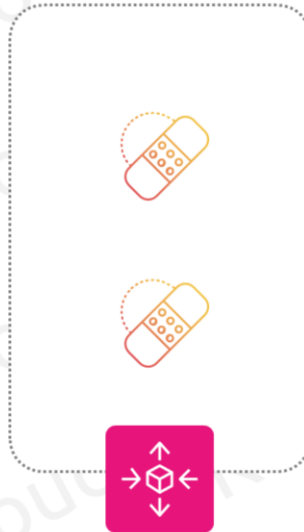
1. Auto Healing: When a EC2 instance is not health, auto scaling group will terminate the instance and launch a new one to match the configuration.

Autoscaling – Manual Fixed Limit

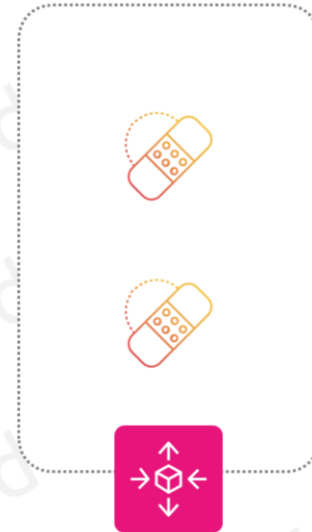
Min: 2 Desired: 2 Max: 2



Min: 1 Desired: 2 Max: 2



Min: 2 Desired: 2 Max: 5



© Copyright KodeKloud

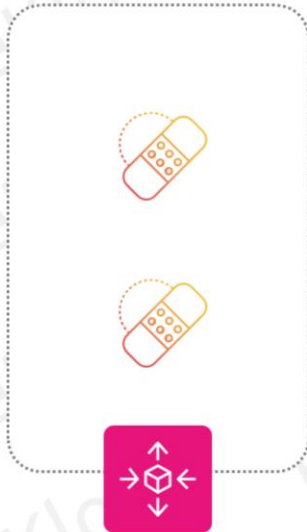
Let's look at the features of auto scaling group and how it is used:

1. **Set Capacity Limit :** Auto scaling requires you to provide three numbers minimum, desired and maximum. Auto scale will make sure minimum number of instance is always running and if desired higher than minimum, then AGS will make sure desired capacity is satisfied.

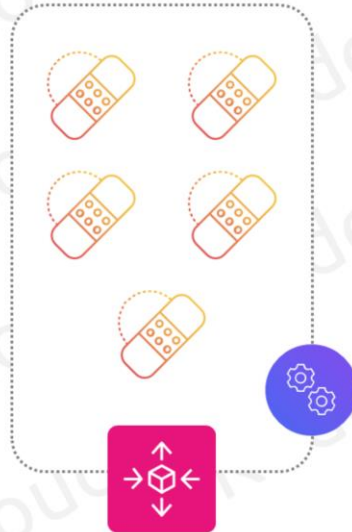
2. If maximum is higher than desired, new EC2 instance are not spined up, it is a threshold for AGS. In the next section we will see how these three parameters influence scaling.

Autoscaling – Manual Desired Capacity

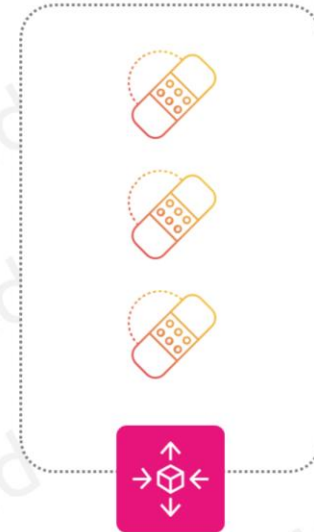
Min: 2 Desired: 2 Max: 10



Min: 1 Desired: 5 Max: 10



Min: 2 Desired: 3 Max: 10

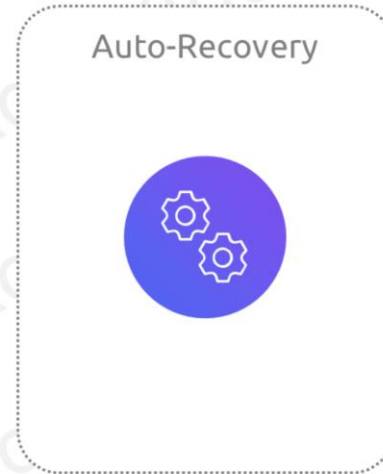
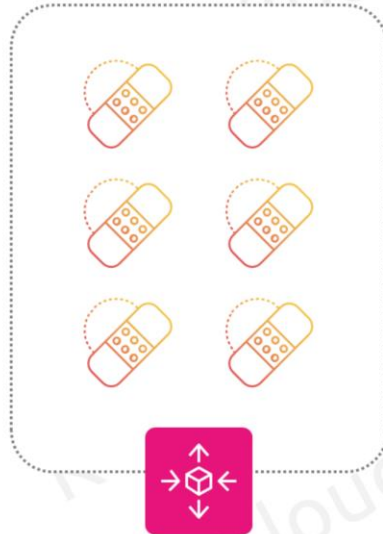


Let's look at the features of auto scaling group and how it is used:

1. **Desired Capacity** : The size of an Auto Scaling group depends on the number of instances that you set as the desired capacity. You can adjust its size to meet demand, by manually increasing desired count. But this is not the recommended way of using auto scaling group.

Autoscaling Group – Auto-Recovery

Min: 1 Desired: 5 Max: 10

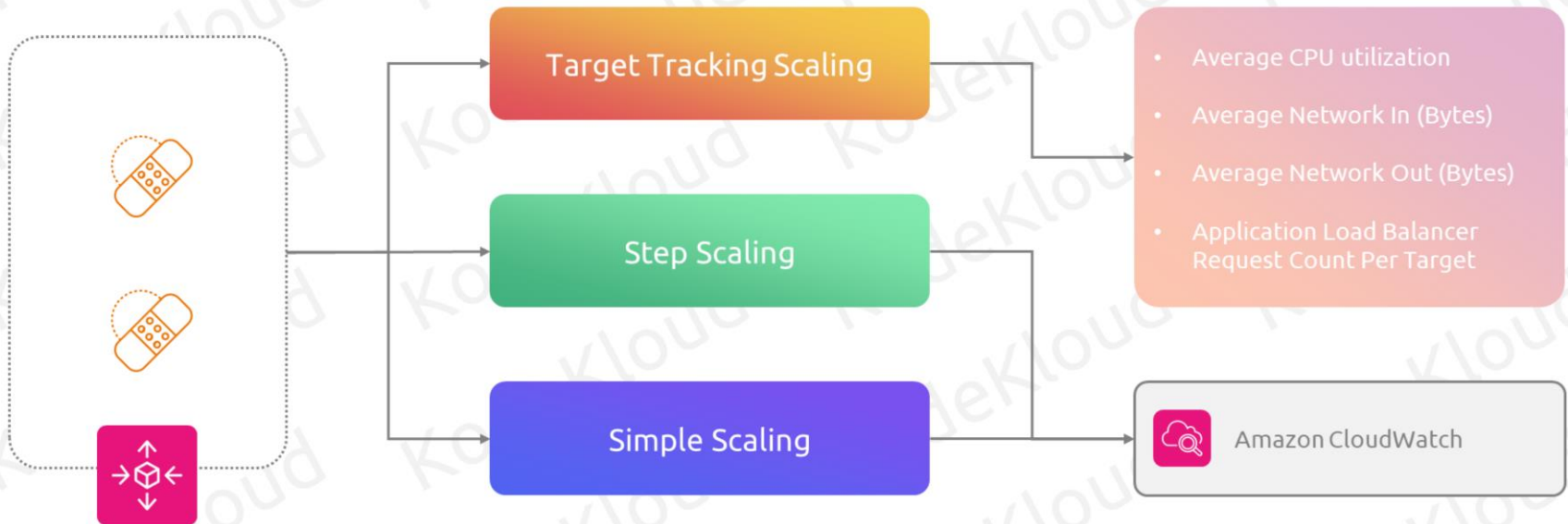


© Copyright KodeKloud

Let's look at the features of auto scaling group and how it is used:

1. **Auto Recovery** : In auto scaling group when one instance is reported to be unhealthy, AGS will terminate the instance and new instance will be launched.

Autoscaling – Dynamic Scaling Policy



© Copyright KodeKloud

Let's look at the features of auto scaling group and how it is used:

1. **Scaling Policy** : When the scaling policy is in effect, the Auto Scaling group adjusts the desired capacity of the group, between the minimum and maximum values that you specify, and launches or terminates the instances as needed.

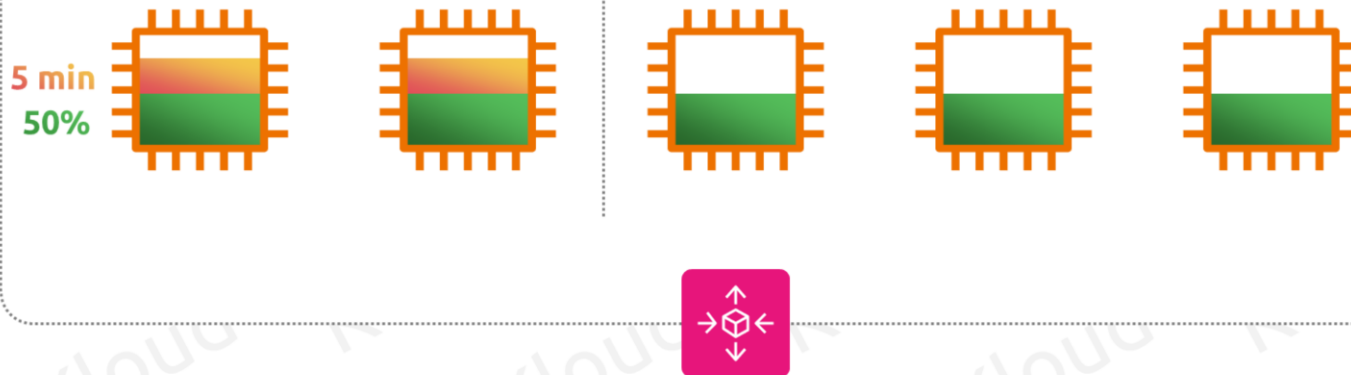
We have three type of Dynamic Scaling Policy:

1. Target tracking Scaling: In this strategy , as the name suggest, a specific metrics is tracked based on that scaling is triggered. AGS by default supports:
 1. Average CPU utilization
 2. Average network In
 3. Average Network out
 4. Application load balancer request count per target (We will be learning about about this in the next lecture about load balancer)
2. Step Scaling and
3. Simple Scaling

Let's look at each of these in detail

Target Tracking Scaling

Min: 1 Desired: 2 Max: 5



© Copyright KodeKloud

Dynamic Scaling – Target Tracking Scaling:

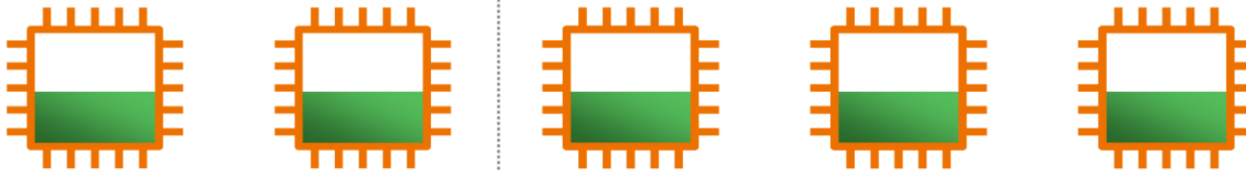
1. One of the four supported metrics is selected, for example let's consider CPU utilization. In target tracking scaling, AGS is waiting for a target to occur, when the average CPU utilization is above the threshold a new instance is added. For example two instances are running and average CPU utilization is less than 50% only two are running. When the usage is above 50% for five minutes a auto scaling launches new instance and new instance will start sharing the load,

resulting in drop of average utilization.

When the newly added EC2 instance is also busy and average usage is more than 50% , another additional instance is added.

Target Tracking Scaling

Min: 1 Desired: 2 Max: 5

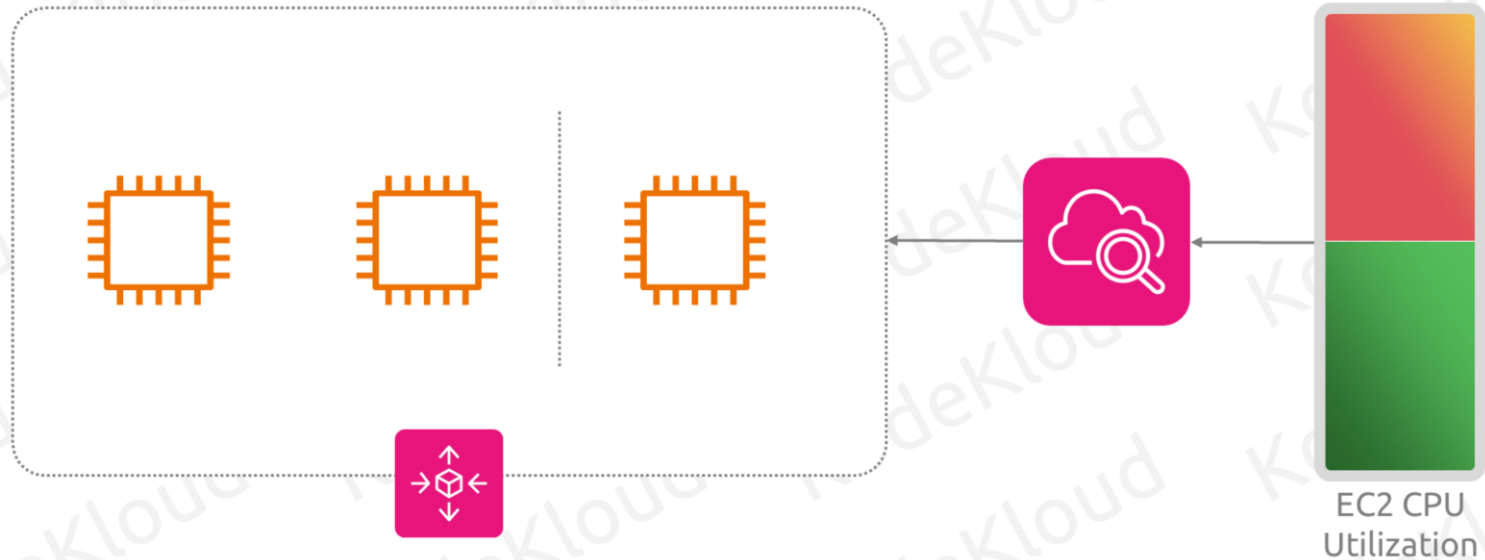


© Copyright KodeKloud

Let's look at the features of auto scaling group and how it is used:

Similarly when the average Utilization drop, the EC2 instances are terminated one by one until the desired capacity is satisfied.

Simple Scaling



© Copyright KodeKloud

Let's look at the features of auto scaling group and how it is used:

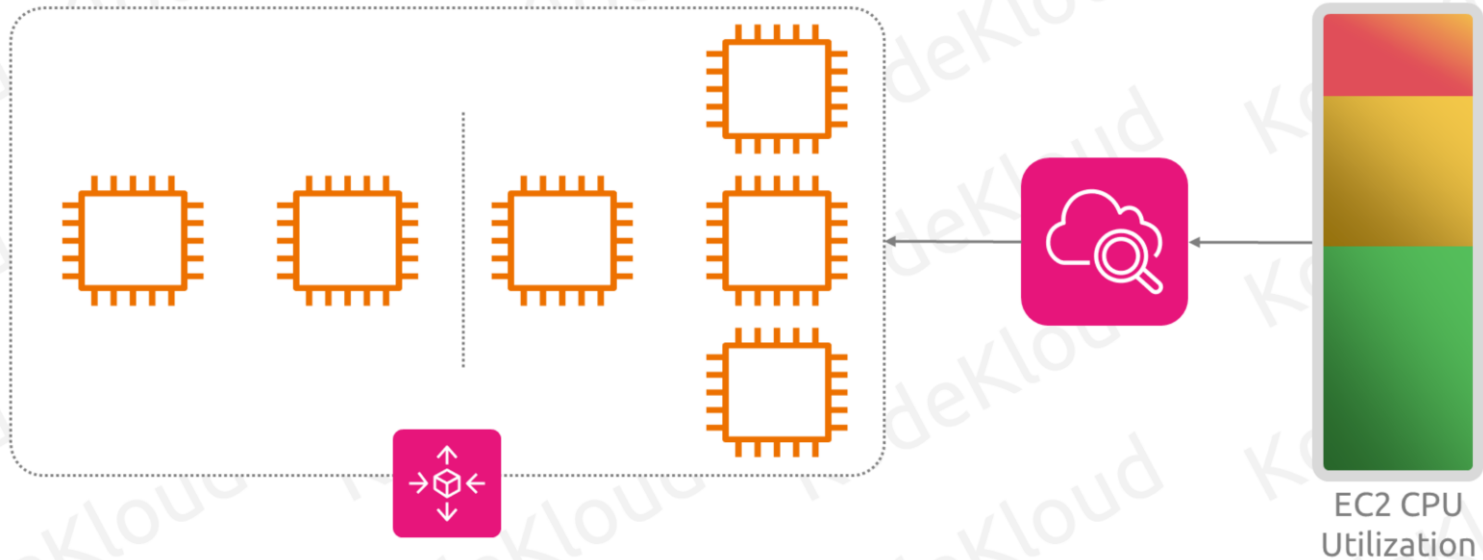
Simple Scaling: This another scaling strategy for fixed conditions and for simple scaling to work we have to create cloudwatch alarms,

1. One for scaling-up
2. One for scaling-in

When the scaling-up alarm is triggered, scaling happens based on what is defined in the policy, we have options to add a certain number of instances, set to a certain number of instances. CloudWatch alarm can be created for any AWS service for example: S3, if the number of objects increase above a threshold then trigger an alarm, to scale EC2 instances which can process faster.

Similarly: when a lower usage alarm is triggered EC2 instances can be scaled-in, similar we can delete X number of instances and or set to a specific count.

Step Scaling



© Copyright KodeKloud

Let's look at the features of auto scaling group and how it is used:

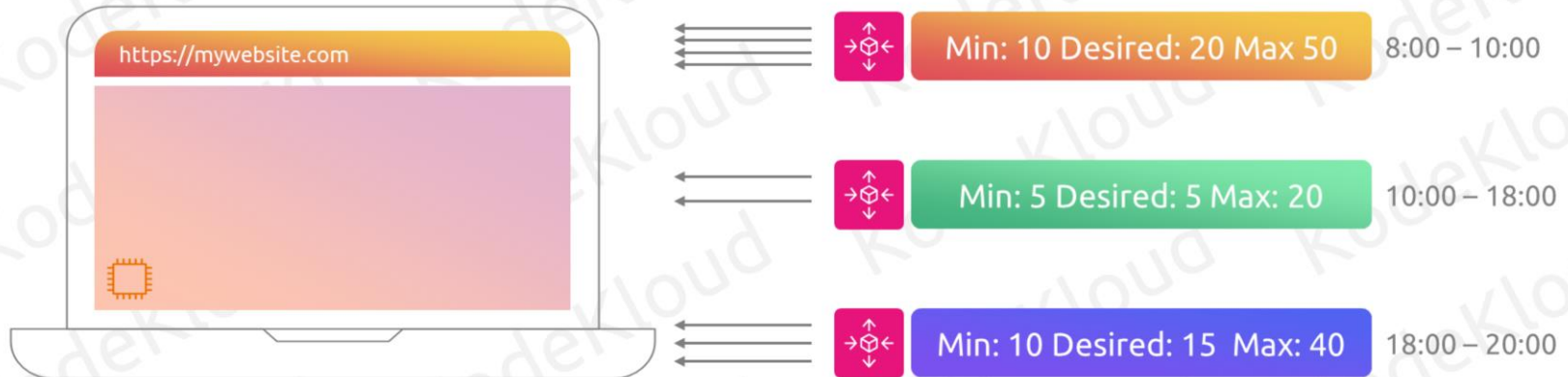
Simple Scaling: This another scaling strategy for fixed conditions and for simple scaling to work we have to create cloudwatch alarms,

1. One for scaling-up
2. One for scaling-in

When the scaling-up alarm is triggered, scaling happens based on what is defined in the policy, we have options to add a certain number of instances, set to a certain number of instances. CloudWatch alarm can be created for any AWS service for example: S3, if the number of objects increase above a threshold then trigger an alarm, to scale EC2 instances which can process faster.

Similarly: when a lower usage alarm is triggered EC2 instances can be scaled-in, similar we can delete X number of instances and or set to a specific count.

Autoscaling – How Schedules Scaling Works?



© Copyright KodeKloud

Let's look at the features of auto scaling group and how it is used:

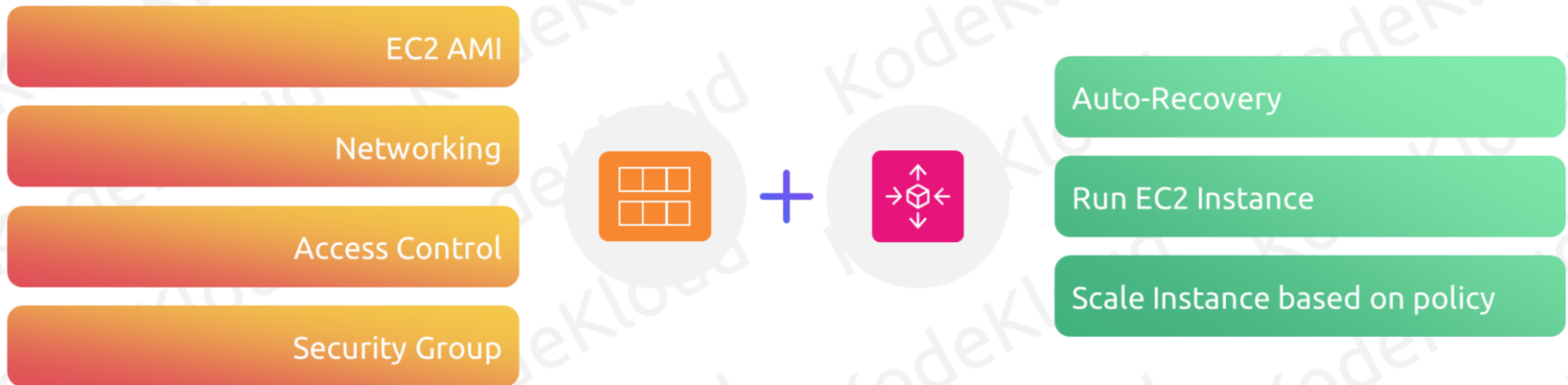
Lets try to understand scheduled scaling with an example of, lets assume a website is running on ec2 instance it receives traffic majority of the traffic between 8:00- 10:00 and 18:00 – 20:00, traffic between 10:00 – 18:00 is very nominal.

If the scale requirement hours are known then schedules scaling would meet the requirement.

Let's look at the features of auto scaling group and how it is used:

1. **Scaling Policy** : When the scaling policy is in effect, the Auto Scaling group adjusts the desired capacity of the group, between the minimum and maximum capacity values that you specify, and launches or terminates the instances as needed.
1. **Scheduled scaling policy** can be triggered once, or by using cron it can be designed to run on a daily basic.

EC2 Autoscaling

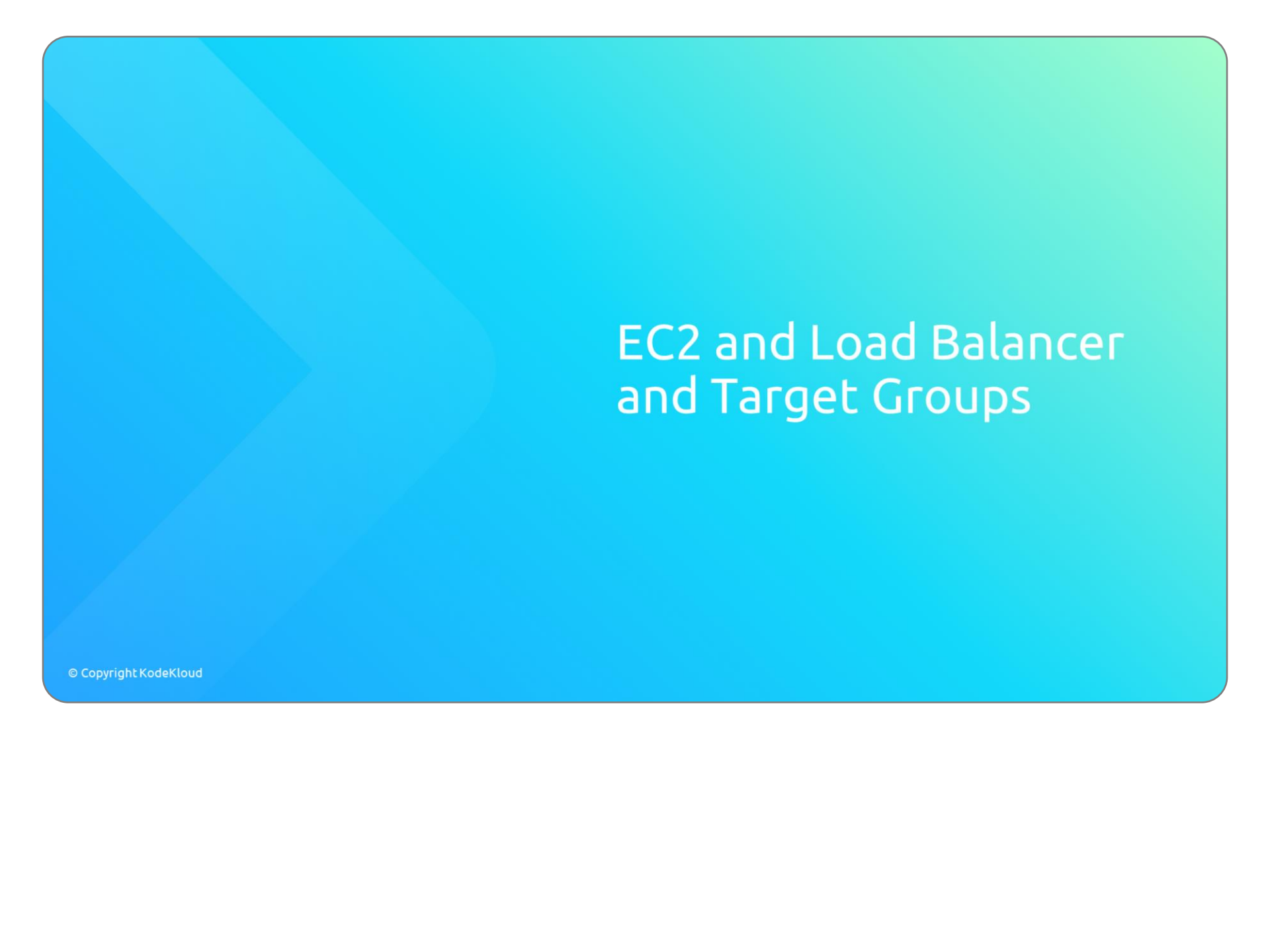


© Copyright KodeKloud

Working of EC2 Auto Scaling group.

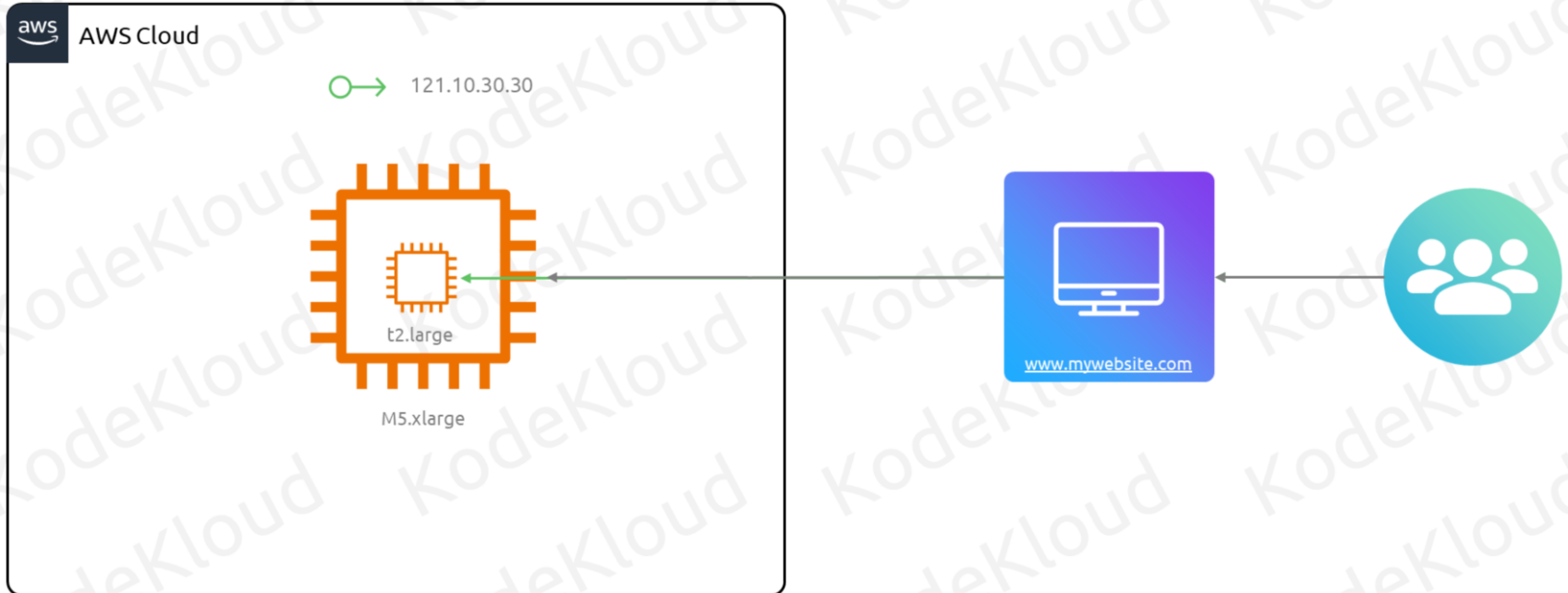
1. When we create an auto scaling group, a launch template is mandatory:
2. Launch Template: is responsible for EC2 instance AMI, networking, access control, security group etc
3. Auto scaling group is only responsible for making sure the desired number of EC2 instances are running and it can scale based on the demand.

That's all in this lecture, lets practice.



EC2 and Load Balancer and Target Groups

Why Do We Need Load Balancer?



© Copyright KodeKloud

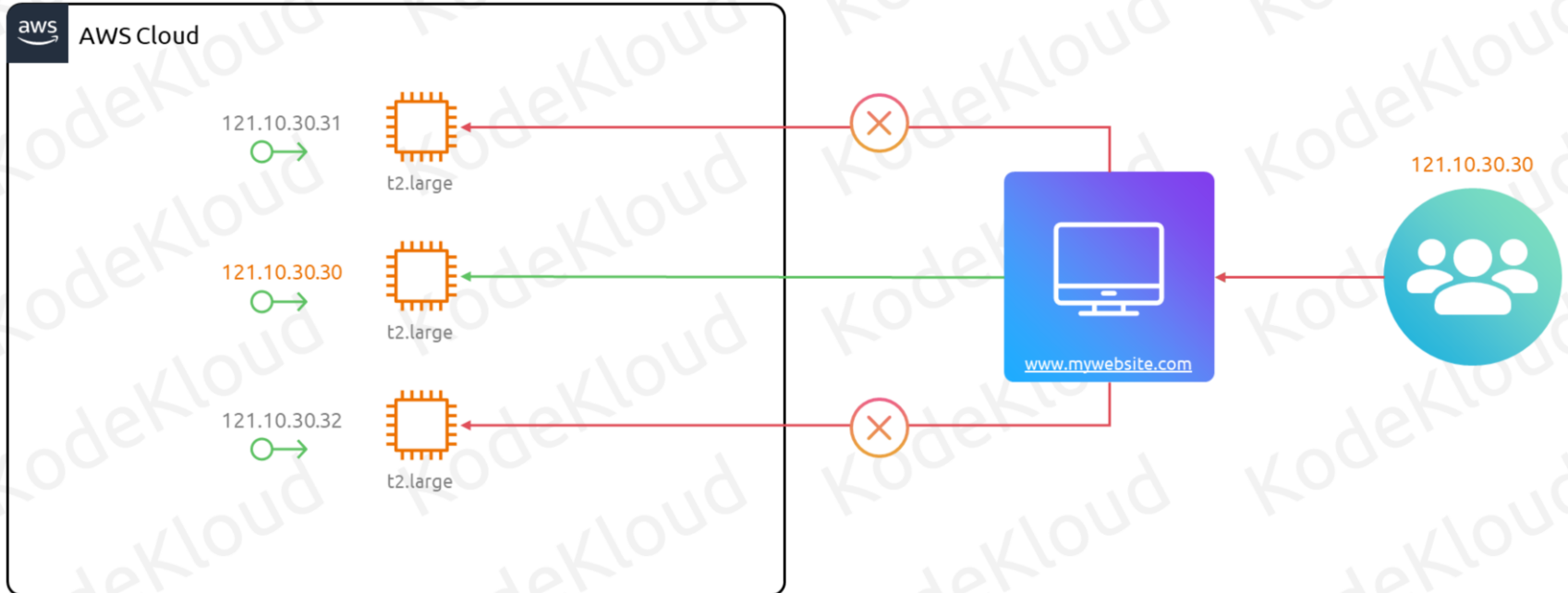
Let assume we are running a website called mywebsite.com on EC2 instance.

When we start the application can be in a single EC2 instance and as the website traffic grows, single instance which we started will not be enough, to meet the demand, the server should also grow,

We have two ways to scale

1. Vertical : increasing the size of the EC2 instance and we have size limit to which we can grow. This also means the server is always scaled up irrespective of traffic. This is not a efficient way to scale/

Why Do We Need Load Balancer?

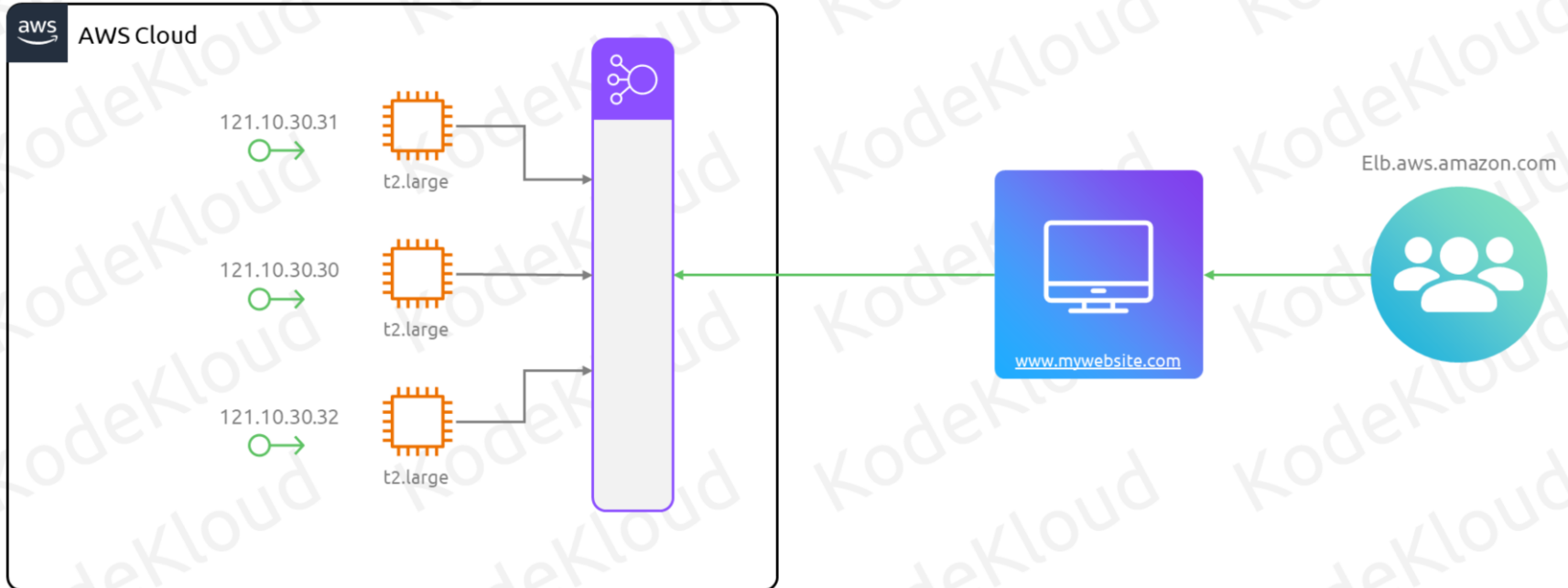


© Copyright KodeKloud

1. Let's look at > one more example > >of scaling.
2. The application "mywebsite.com" is hosted on a t2.large ec2 instance which has a public IP 121.10.30.30. And users can access to application.
3. > But when application uses increase , a single instance is not enough to handle the incoming traffic.
4. > To over come this limitation, instead of upsizing the exisiting instance,> multiple instance are launched with same application running on the EC2 instance. This is called as horizontal scaling, we add new instance as the traffic grows.

But As you can see the domain mywebsite.com is pointed to a single EC2 instance, even after new instances are added traffic is not deviated to the newly launched EC2 instance. To overcome this limitation a load balancer can be used which will be a single point of contact for the clients irrespective of number of EC2 instance runs behind.

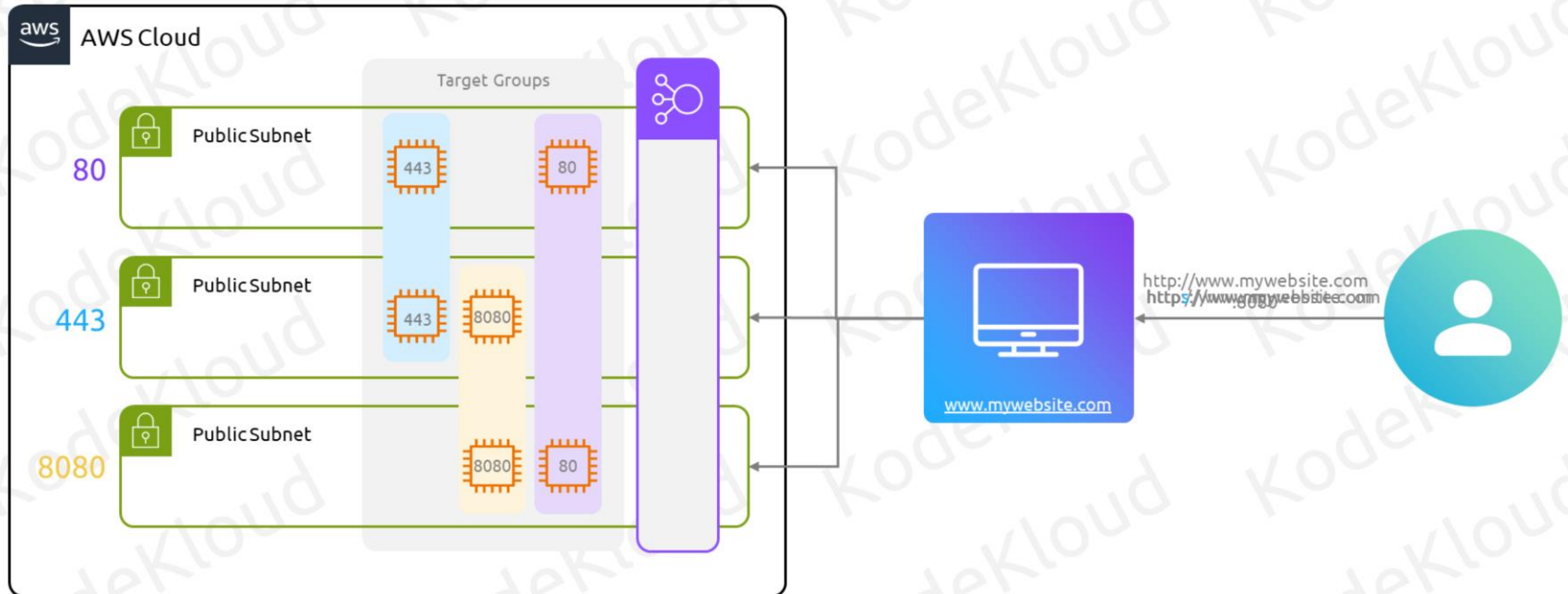
Why Do We Need Load Balancer?



© Copyright KodeKloud

1. let's look at the same example but with a load balancer instead of IP address
2. > All users traffic >> lands on > elastic load balancer, and a single EC2 instance is registered to the load balancer.
3. > As the traffic grows new EC2 instances are registered with the load balancer automatically, the scaling process is completely transparent to the user.

How Does a Load Balancer Work?



© Copyright KodeKloud

Let's look at the overview of how LB works in AWS before learning about different type of loadbalancer supported by AWS and individual features.

AWS Elastic Load balancer is a highly saleable and fault tolerant, load balancer spans across different availability zone for HA.

EC2 instances are grouped together based on specific characteristics and this grouping is called "target groups", we will learn about this in the following lectures and hands on lab.
Same EC2 instance can be part of two target groups.

1. Listener:
 - > Every load balancer has a listener, which receives the traffic. In the current flow port number 80 is the listener, and traffic is forwarded to the configured target group.>
2. > HTTPS Traffic
 - terminated on 443 listener by default.>
3. > Load balancer also supports, custom ports like 8080.

When traffic arrives on the load balancer, listen rules are evaluated based on the priority order, when a match is found traffic is forwarded to the target group which is configured in the rule.

Load balancer can be public, where public clients interact directly or private: where the request can originate from another set of EC2 instances within a VPC.

The main purpose of load balancer is for scaling and high availability.

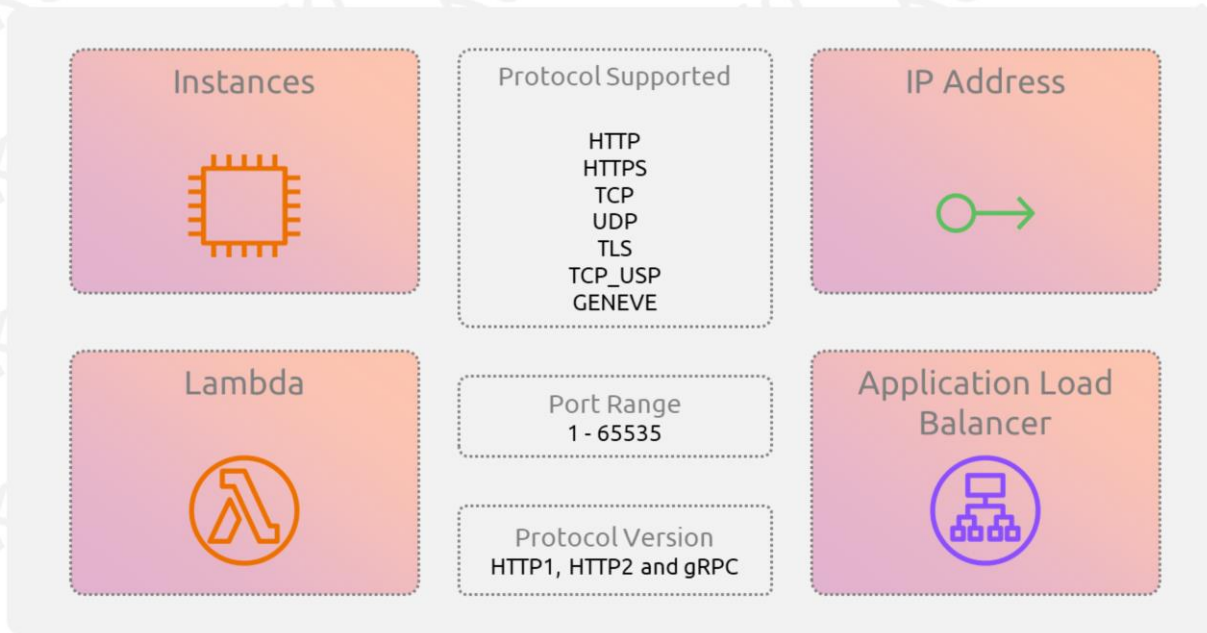
What Is Target Group?



© Copyright KodeKloud

Group EC2 instances can be considered as target and it is grouped into target group based on type of traffic it can serve. AWS target group routes request incoming request from load balancer to one or more registered targets.

Target Group Characteristics



© Copyright KodeKloud

AWS target group supports following targets,

1. EC2 instance
2. IP address
3. Lambda Function
4. Application Load balancer

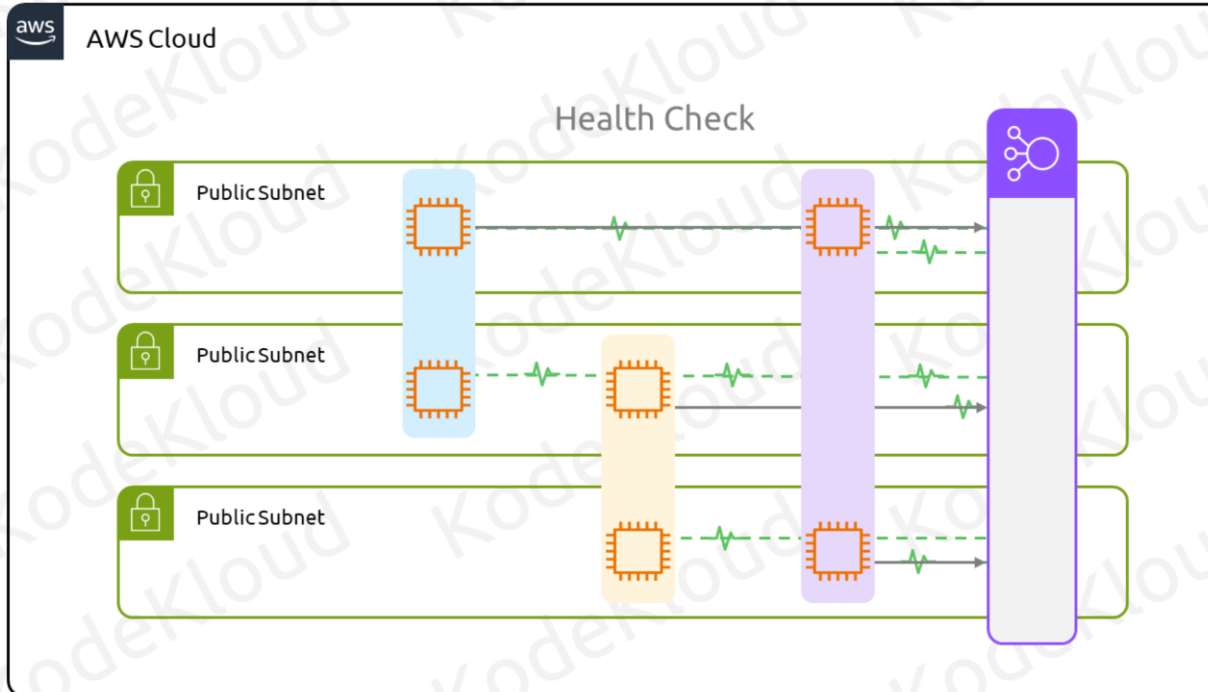
Supported protocols are

1. HTTP
2. HTTPS
3. TCP
4. UDP
5. TLS
6. TCP_USP
7. GENEVE

Port range from 1 – 65535

Protocol version HTTP1, HTTP2 and gRPC

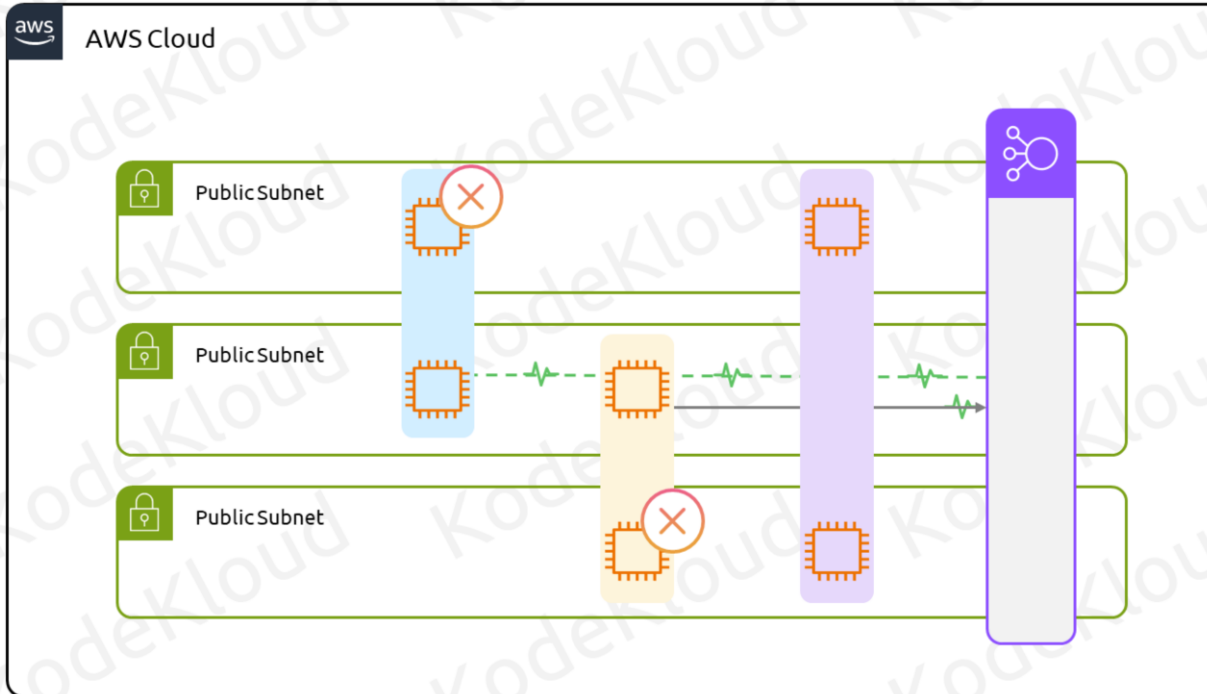
Load Balancer Health Check



© Copyright KodeKloud

Load Balancer node always performs health check on the EC2 instance which is registered to the load balancer at a regular interval which is configured while creating a target group.

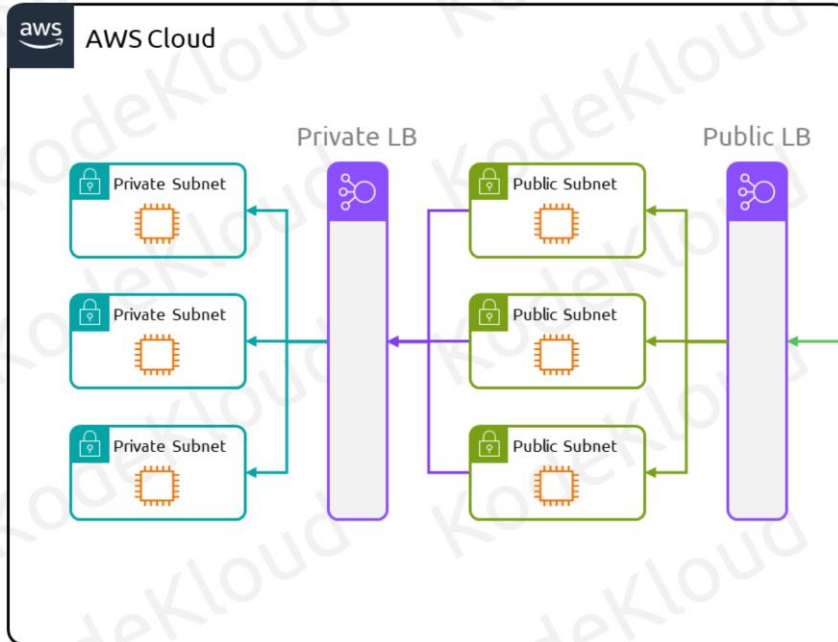
Load Balancer Health Check



© Copyright KodeKloud

As soon as health check fails of a EC2 instance, load balancer will stop sending traffic to the unhealthy host and only available health host will receive the traffic.

Public and Private Load Balancer

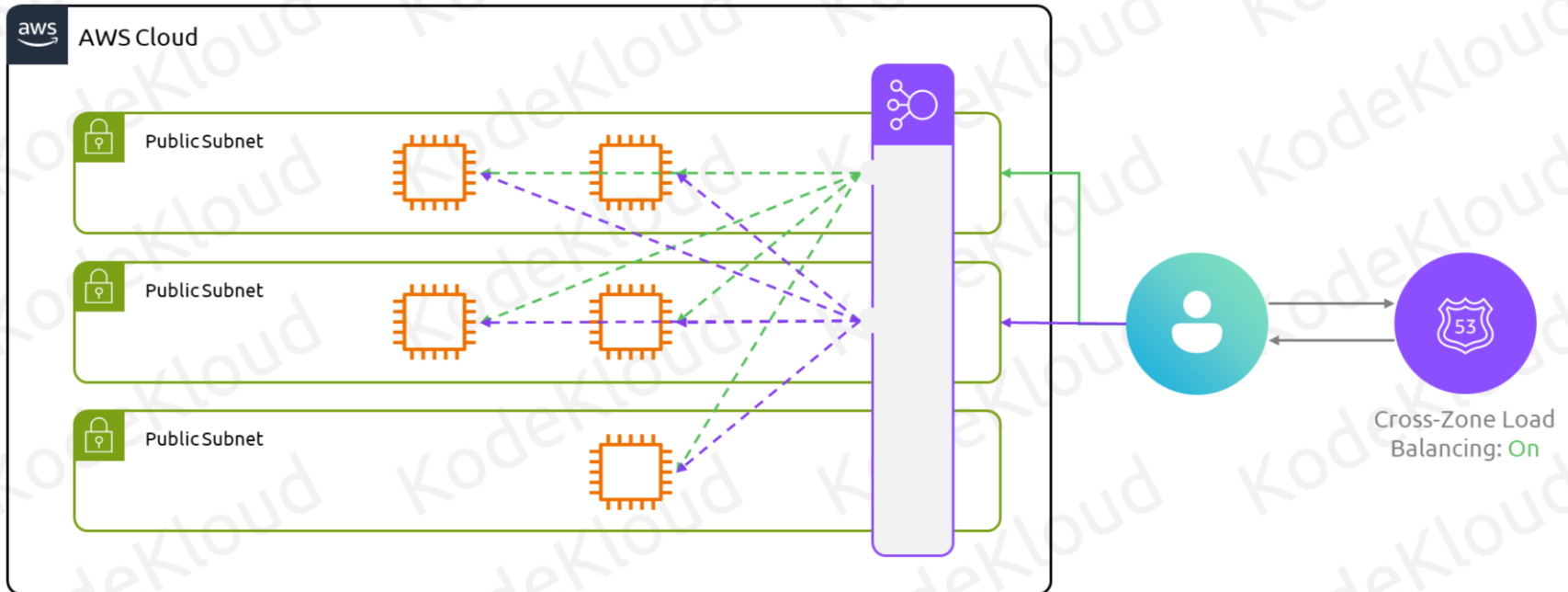


© Copyright KodeKloud

Load balancer can be public, where public clients interact directly or private: where the request can originate from another set of EC2 instances.

As you can see, load balancer is used when you want to scale a system or just a component.

What Is Cross-Zone Load Balancing?



© Copyright KodeKloud

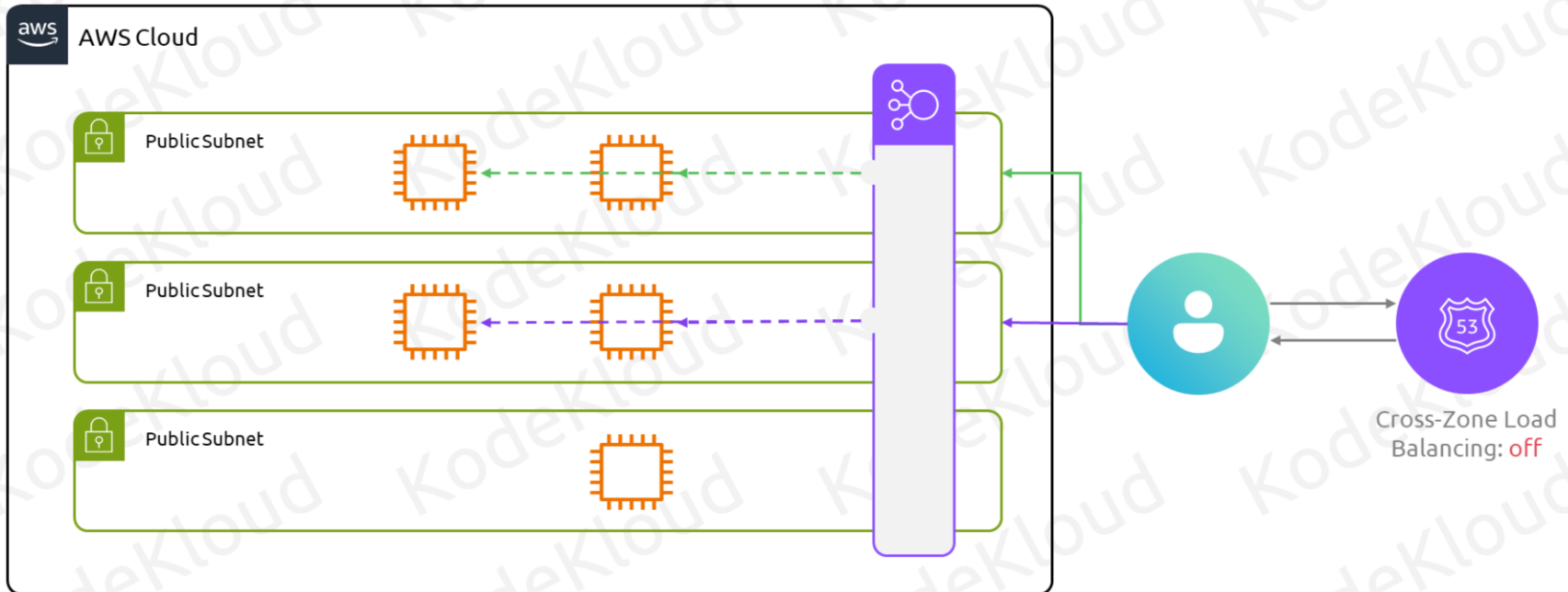
Cross-Zone load balancing is a technique of distributing load across the targets which are registered to the load balancer.

We have EC2 instance spread across three AZ's , when client performs DNS resolution of a domain which is registered to a load balancer, one of the load balancer node IP is provided as response and client connects to one of the node. When cross-zone load balancing is enabled, a load balancer node can send traffic to each and every EC2 instance which is

registered to the load balancer, irrespective of the AZ in which the load balancer node is present. also the traffic will be forwarded to ec2 in an AZ where load balancer node is not running.

This ensure that each and every EC2 instance received equal amount of traffic and a single AZ is never over loaded.

What Is Cross-Zone Load Balancing?



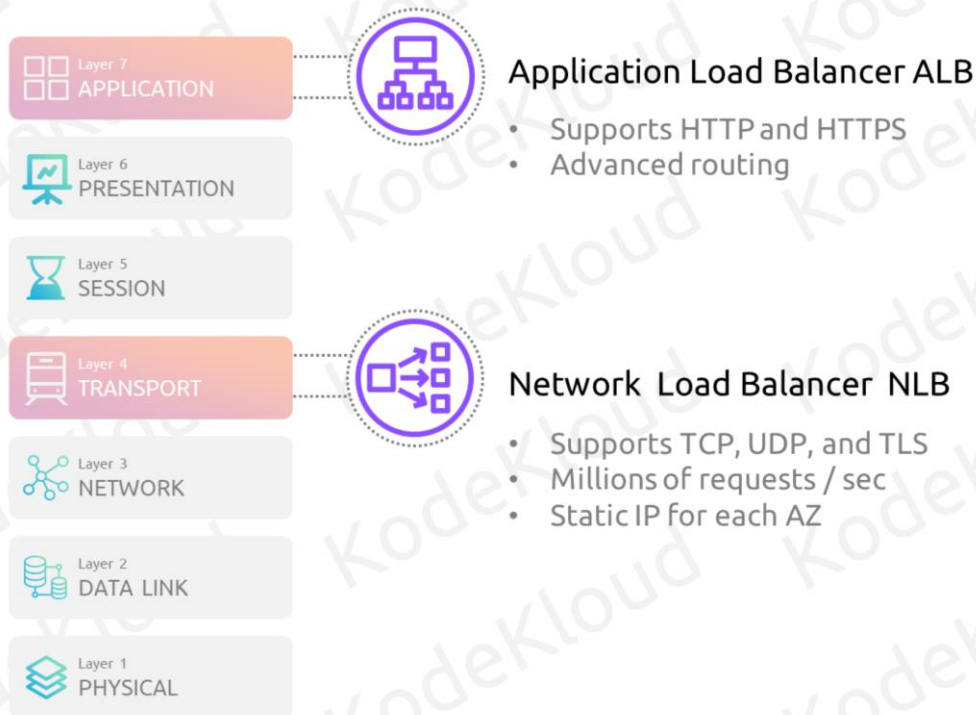
© Copyright KodeKloud

When cross-zone load balancing is turned off, the traffic which is received by the load balancer node will forward it to only the EC2 instance which are present in the respective AZ, it does not forward the traffic across AZ's.

This may result in uneven traffic distribution across AZ's, AZ in which EC2 instance are less will have high utilization. And if we have a target in a AZ where load balancer node is not present then that instance will never receive any traffic.

It is recommended to keep cross-zone load balancing enabled always.

Types of LB Supported by AWS



© Copyright KodeKloud

AWS Supports two type of load balancers, application and network load balancer.

Application Load balancer / ALB

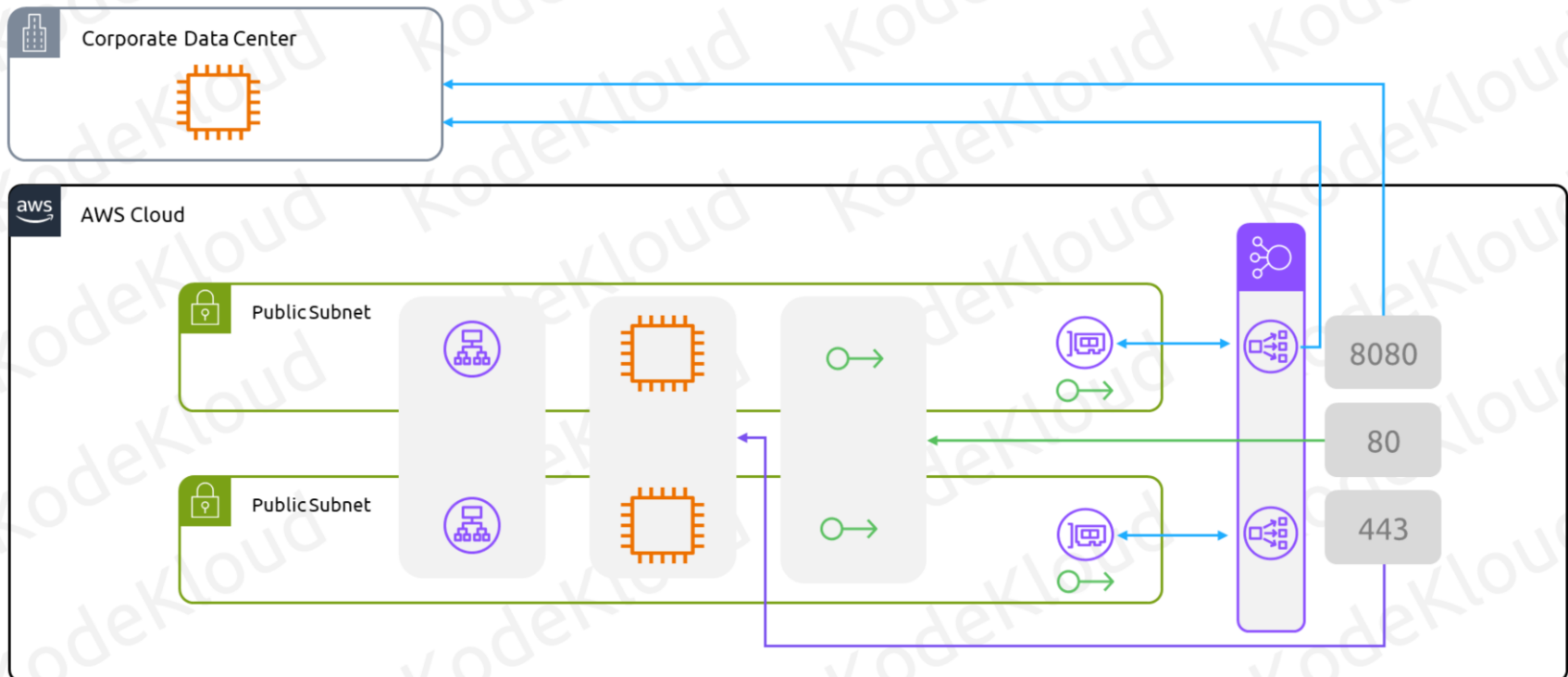
- Application Load Balancer functions at the application layer, the seventh layer of the Open Systems Interconnection (OSI) model.

- ALB supports HTTP and HTTPS Traffic
- Support advanced routing traffic

Network load Balancer / NLB

- A Network Load Balancer functions at the fourth layer of the Open Systems Interconnection (OSI) model.
- NLB supports TCP, UDP, TLS
- It can handle millions of requests per second
- A Static IP can be configured for each AZ in which the NLB node is present, allowing clients to have more control on the routing.

How Does NLB Work?



© Copyright KodeKloud

NLB Supports following Target Groups

When you create a target group, you specify its target type,, you can register

- instance IDs,
- IP addresses, or
- an Application Load Balancer.

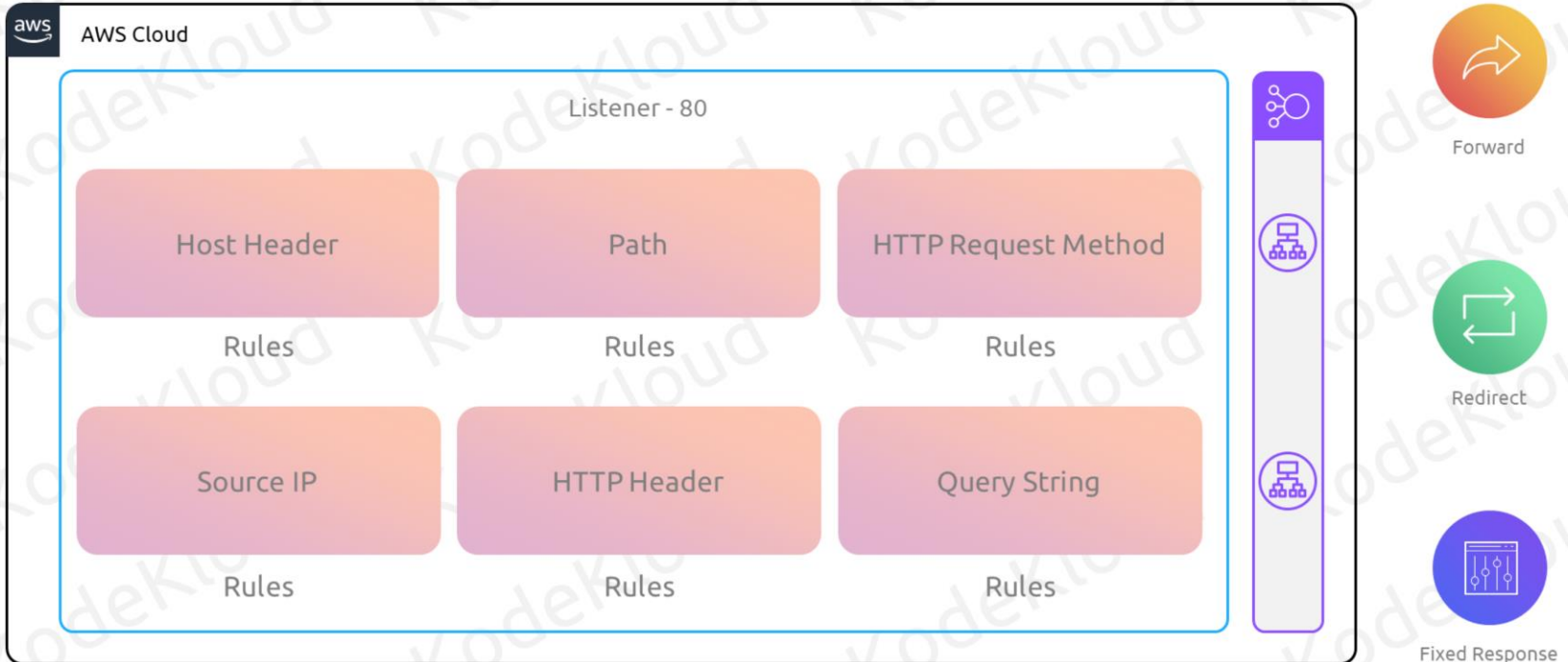
When we create NLB a network interface is created for each Availability Zone, Each load balancer node in the Availability Zone uses this network interface to get a static IP address. When you create an Internet-facing load balancer, you can optionally associate one Elastic IP address per subnet.

A listener is created which will forward the traffic to the selected target group. For example:

1. Listener 80 will forward the traffic to target group with IP address
2. Listener 443 will forward the traffic to target group with instances registered
3. NLB also supports forwarding client request out VPC network like, corporate data center which is connected by VPN or direct connect.

As NLB operated in layer 4, it supports simple listener rules but it is highly saleable, elastic IP for AZ is a feature which is supported only by NLB, this is normally helpful for whitelisting.

Feature of ALB



© Copyright KodeKloud

Lets look at feature of ALB,

A listener is configured to receive traffic, lets take port 80 for example.

A listener contains rules and each rules can support few conditions, which is the strength of ABL. Each rule must include

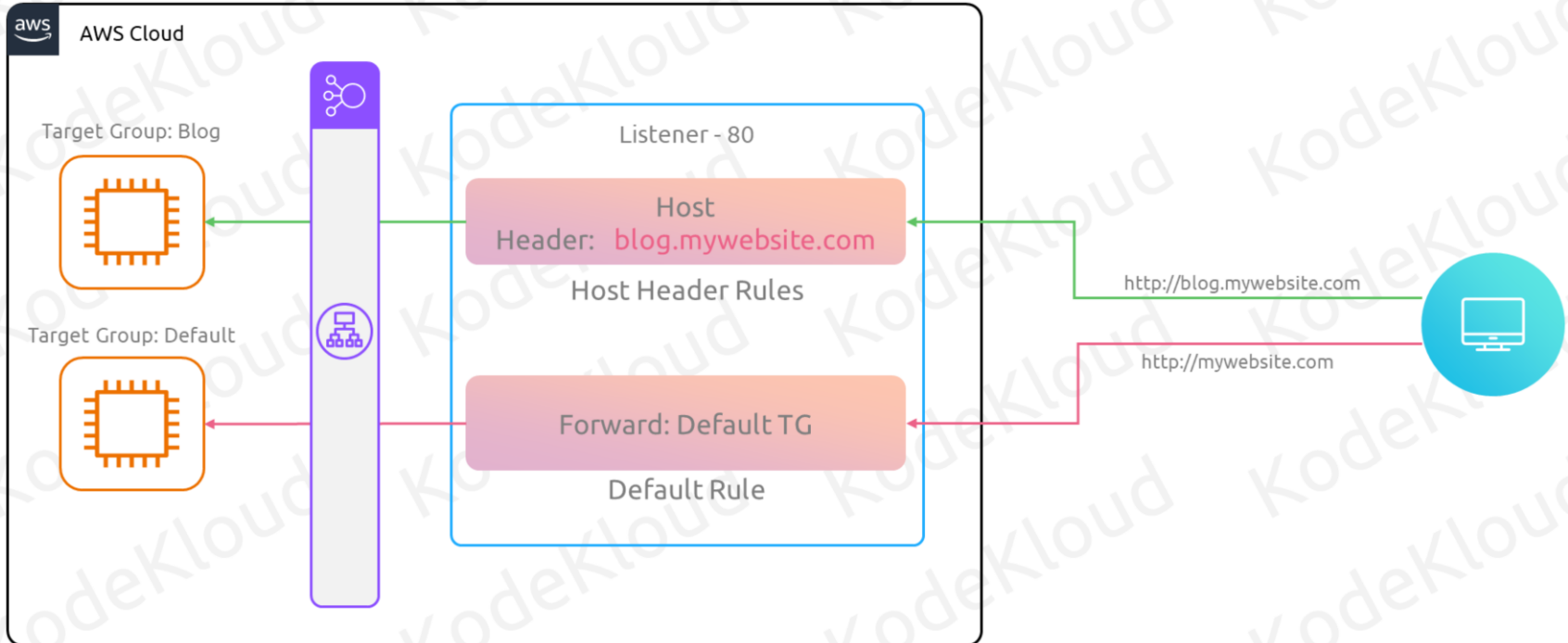
exactly one of the following actions: forward, redirect, or fixed-response,

Supported conditions are

1. Host header
2. Path
3. HTTP request method
4. Source IP
5. HTTP Header
6. Query String

Let's look at each of the conditions

ALB Host Header



© Copyright KodeKloud

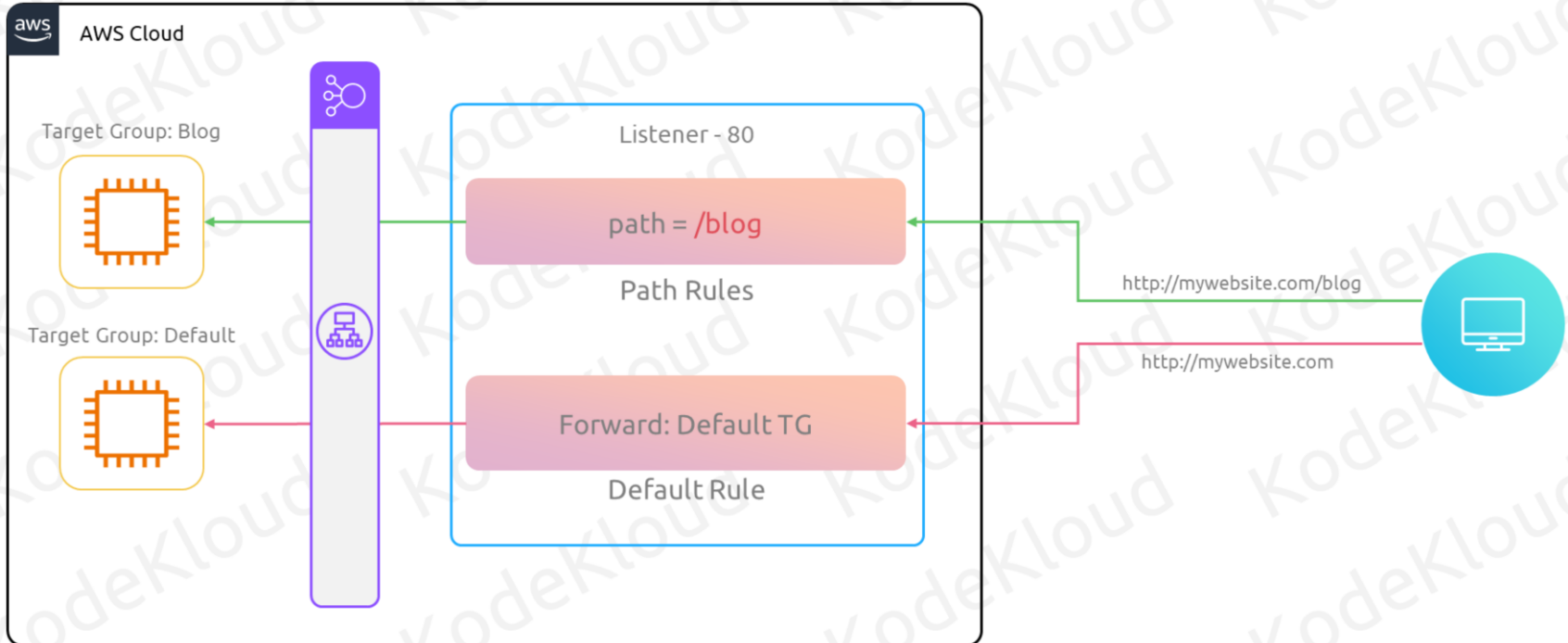
Lets look at host header condition

On ALB A listener is configured to receive traffic, lets take port 80 for example.

A listener is configured with two rules

1. Host header rule : with a condition to match is the host requested is blog.mywebsite.com then forward the traffic to target group "blog" when a blog is running
2. Default rule: We always have a default rule which will be executed at the end. In this case host request to mywebsite.com will be forwarded to default target group.

ALB Path



© Copyright KodeKloud

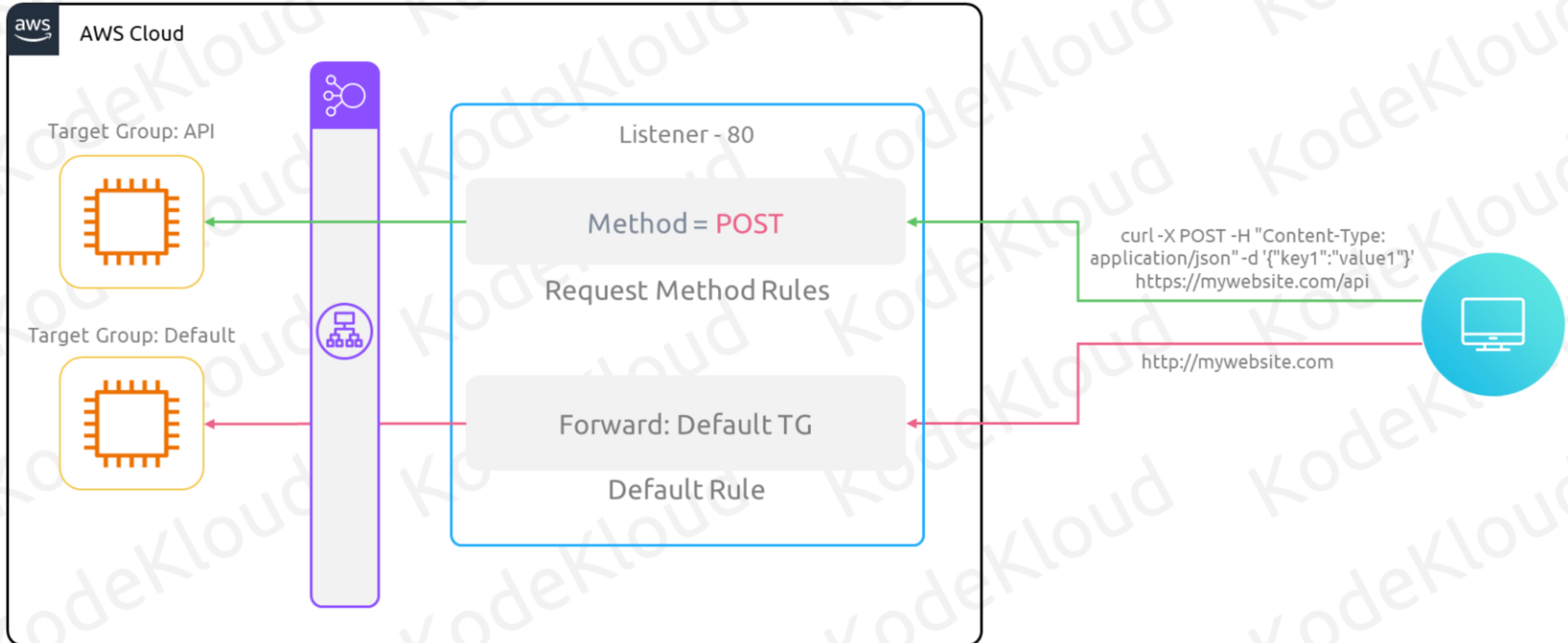
Lets look at path condition

On ALB A listener is configured to receive traffic, lets take port 80 for example.

A listener is configured with two rules

1. Path rule : with a condition to match if the host requested contains path /blog , then forward the traffic to target group "blog" when a blog is running
2. Default rule: We always have a default rule which will be executed at the end. In this case host request to mywebsite.com will be forwarded to default target group.

ALB HTTP Method



© Copyright KodeKloud

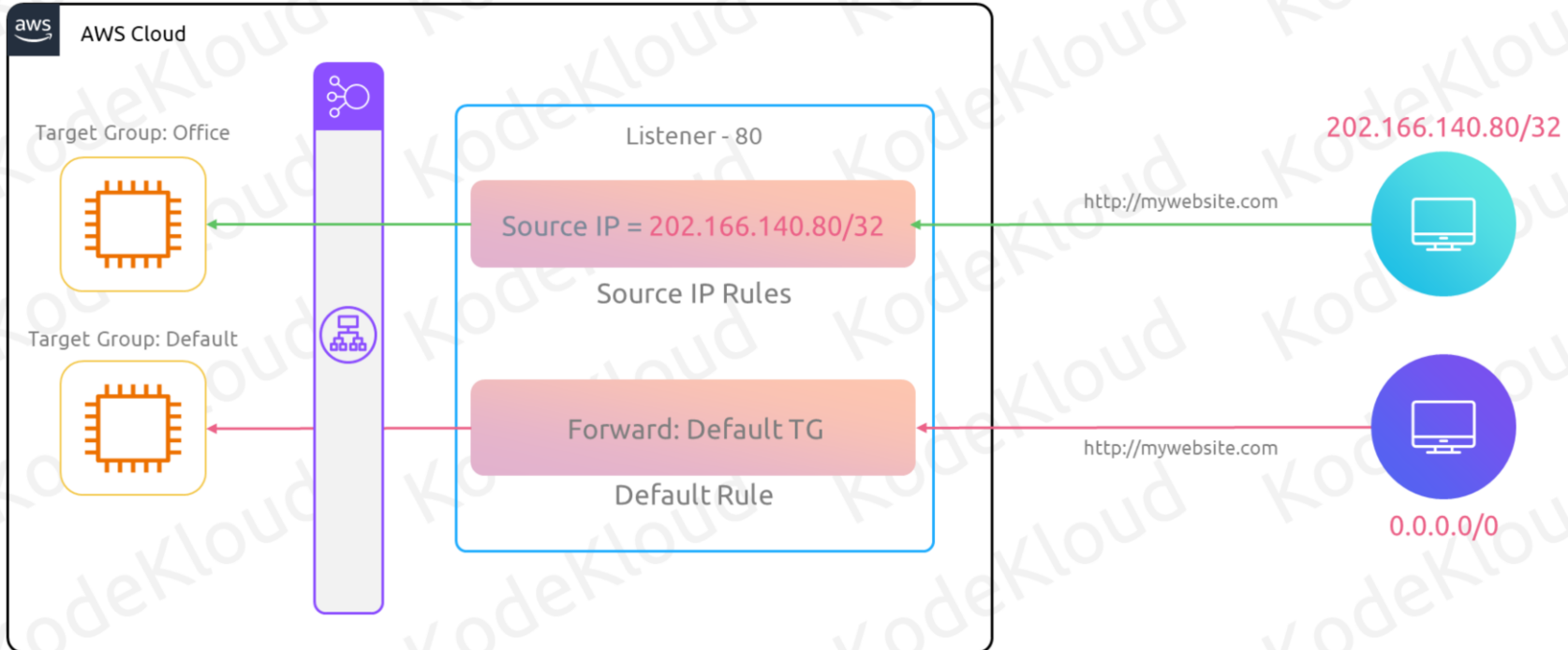
Lets look at HTTP method condition

On ALB A listener is configured to receive traffic, lets take port 80 for example.

A listener is configured with two rules

1. HTTP Method rule : with a condition to match if the request method is POST, then forward the traffic to target group " API" where the request can be processed.
2. Default rule: We always have a default rule which will be executed at the end. In this case any other request method will to mywebsite.com will be forwarded to default target group.

ALB Source IP



© Copyright KodeKloud

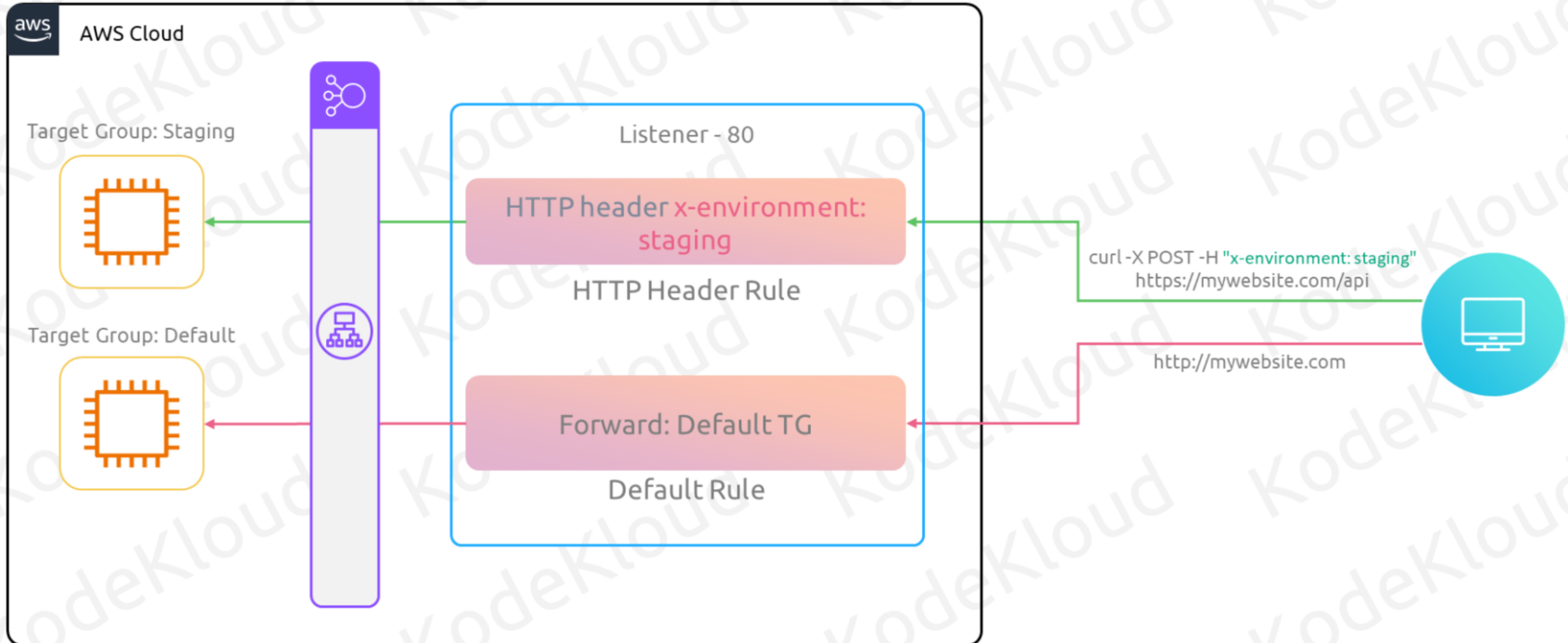
Lets look at HTTP method condition

On ALB A listener is configured to receive traffic, lets take port 80 for example.

A listener is configured with two rules

1. Source IP rule : with a condition to match if the request to "mywebsite.com " originated from a specific IP, then forward the traffic to target group " office" where the request can be processed.
2. Default rule: We always have a default rule which will be executed at the end. In this case any other request method will to mywebsite.com will be forwarded to default target group.

ALB HTTP Header



© Copyright KodeKloud

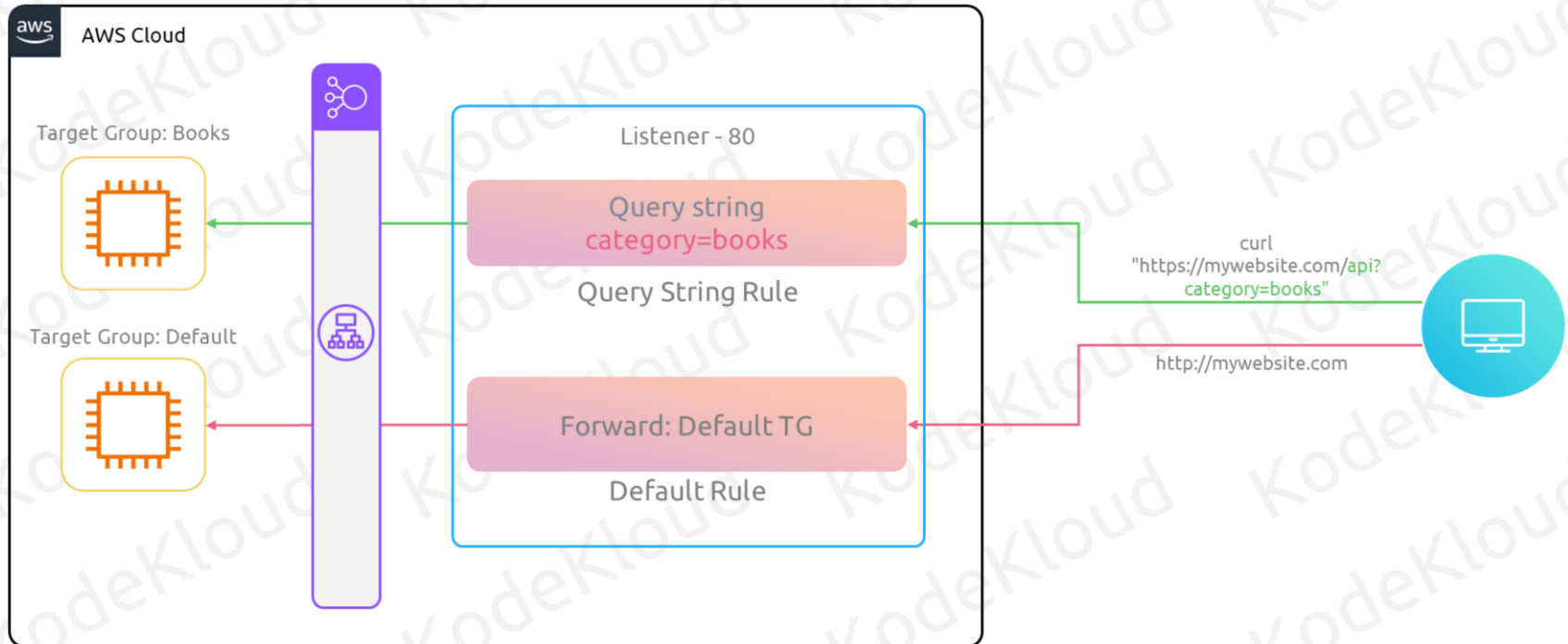
Lets look at HTTP method condition

On ALB A listener is configured to receive traffic, lets take port 80 for example.

A listener is configured with two rules

1. HTTP header : with a condition to match if the request to "mywebsite.com" is made with a header "x-environment: staging", then forward the traffic to target group " staging" where the request can be processed.
2. Default rule: We always have a default rule which will be executed at the end. In this case any other request method will to mywebsite.com will be forwarded to default target group.

ALB Query String



© Copyright KodeKloud

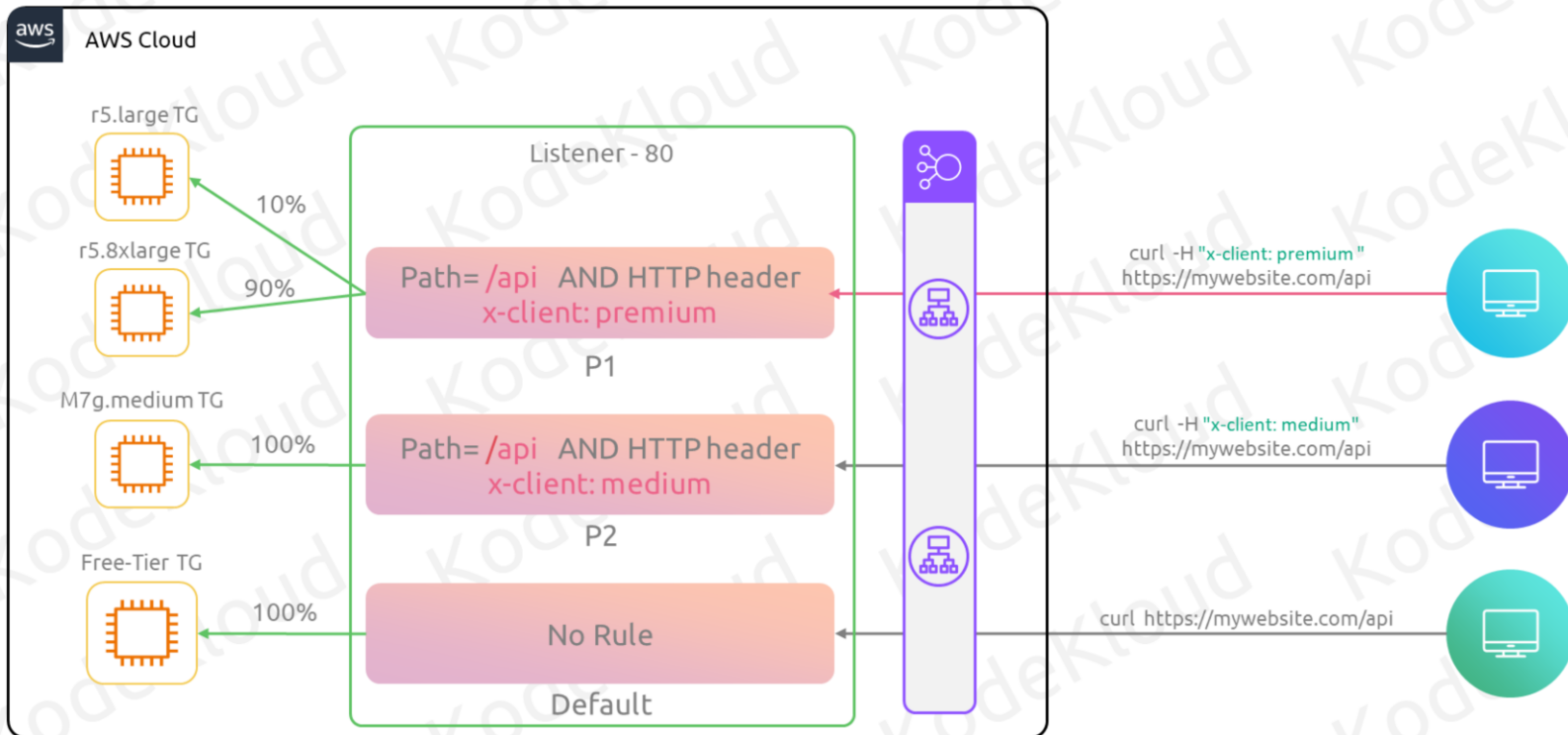
Lets look at query string condition

On ALB A listener is configured to receive traffic, lets take port 80 for example.

A listener is configured with two rules

1. Query String rule: with a condition to match if the request to "mywebsite.com" is made with a query string "category: books", then forward the traffic to target group "books" where the request can be processed.
2. Default rule: We always have a default rule which will be executed at the end. In this case any other request method will to mywebsite.com will be forwarded to default target group.

ALB Rule Execution



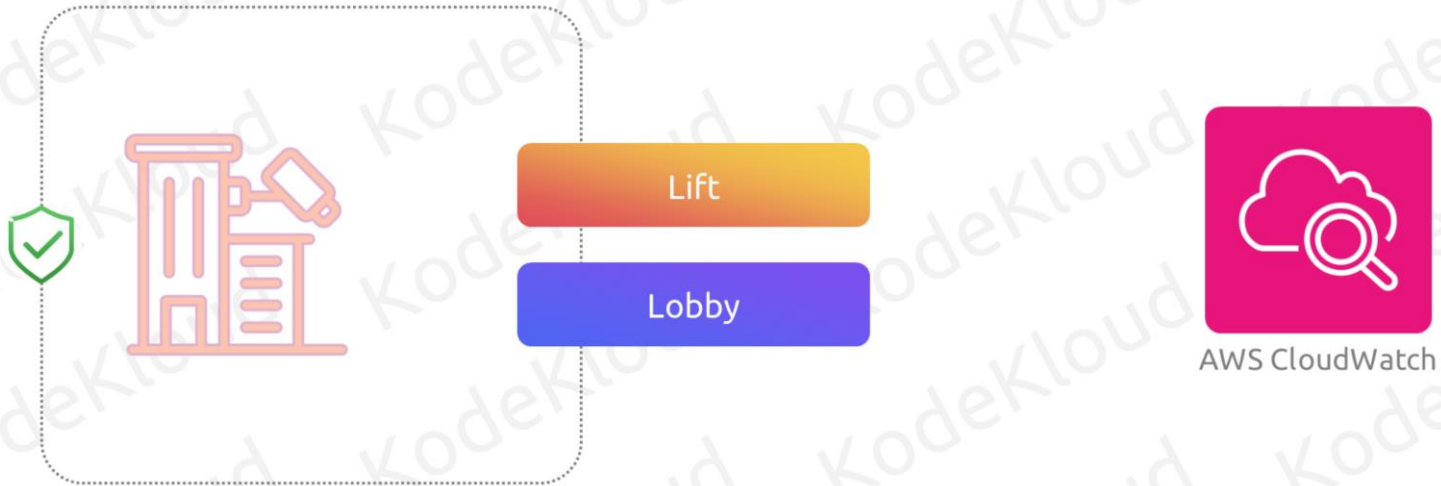
© Copyright KodeKloud

Lets look at rule execution method of ALB,



EC2 CloudWatch and Monitoring

Why Do We Need AWS CloudWatch?

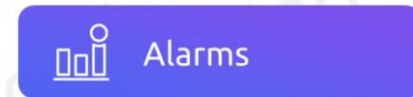
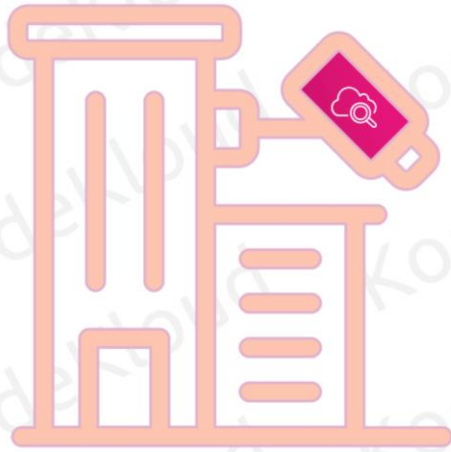


© Copyright KodeKloud

In a building a security system which keeps track of users who are coming in and out of a building, camera used to monitor the activity in the lift lobby and everything in controlled from a single room, which reduces number of human officers required to keep a building safe.

I AWS environment this centralised monitoring service is called AWS cloudwatch.

AWS CloudWatch

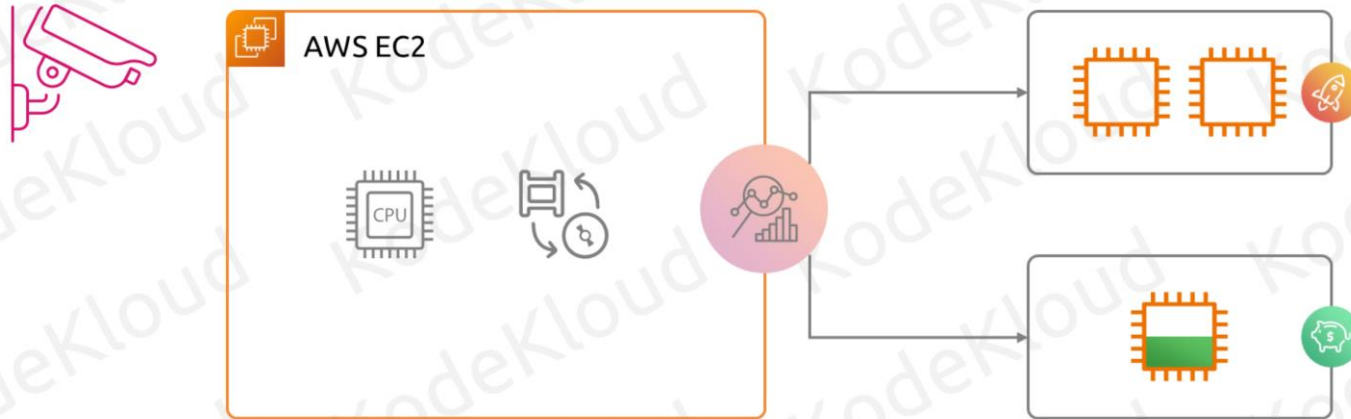


© Copyright KodeKloud

1. Amazon CloudWatch monitors your Amazon Web Services (AWS) resources and the applications you run on AWS in real time.
2. You can use CloudWatch to collect and track metrics, which are variables you can measure for your resources and applications.

3. You can create alarms that watch metrics and send notifications or automatically make changes to the resources you are monitoring when a threshold is breached.

AWS CloudWatch



© Copyright KodeKloud

For example, you can monitor the CPU usage and disk reads and writes of your Amazon EC2 instances and then use that data to determine whether you should launch additional instances to handle increased load. You can also use this data to stop under-used instances to save money.

AWS CloudWatch



Resource
Utilization

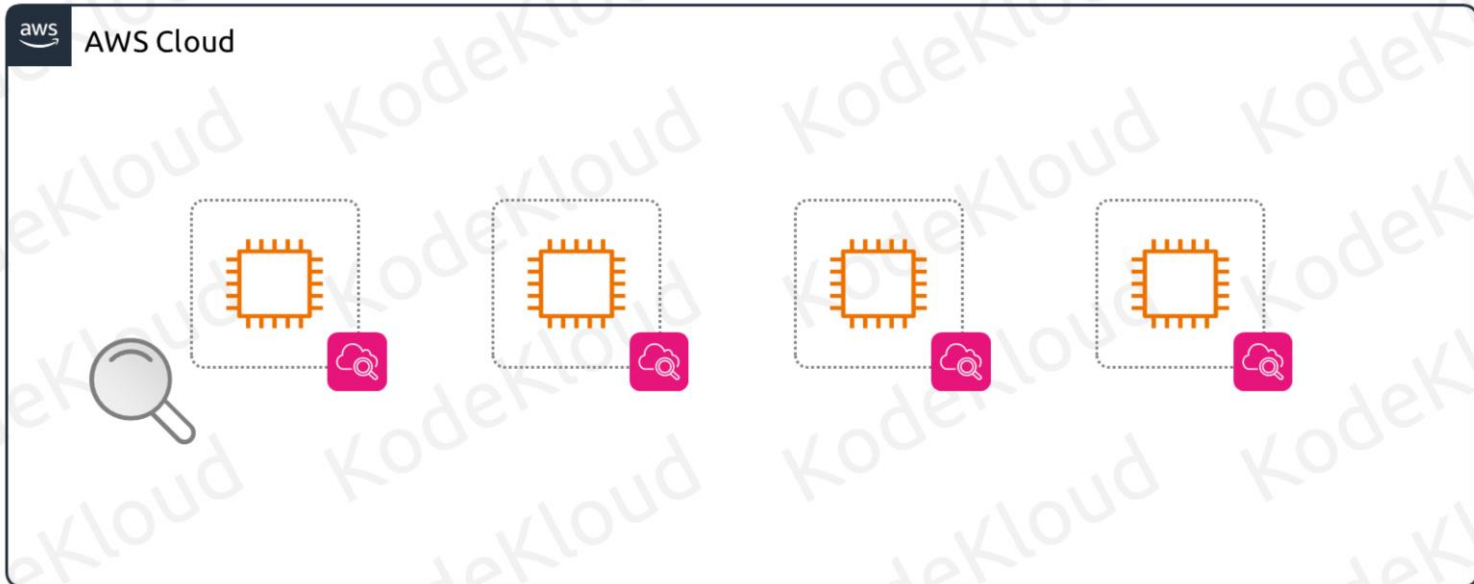
Application
Performance

Operational
Health

© Copyright KodeKloud

4. With CloudWatch, you gain system-wide visibility into resource utilization, application performance, and operational health.

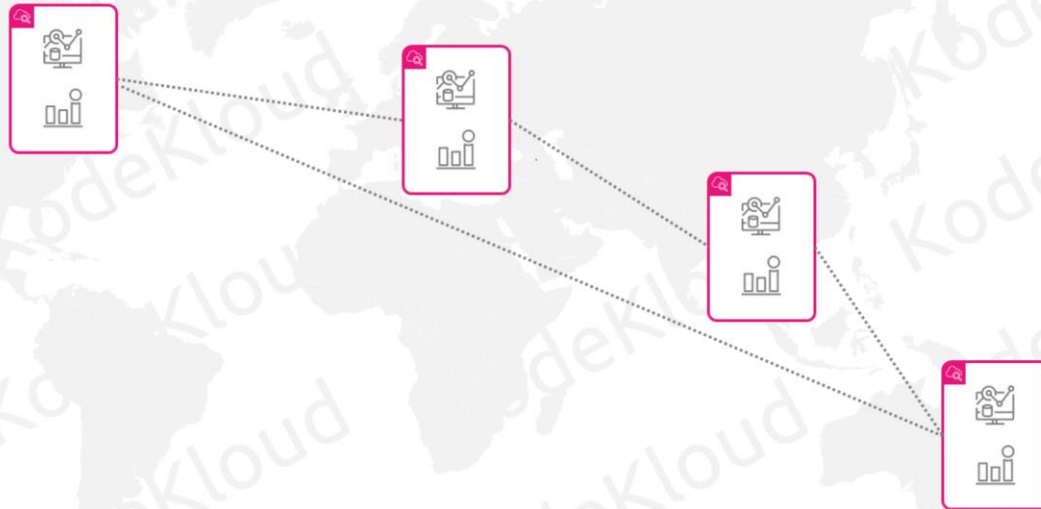
AWS CloudWatch



© Copyright KodeKloud

4. With CloudWatch, you gain system-wide visibility into resource utilization, application performance, and operational health.

CloudWatch Dashboard



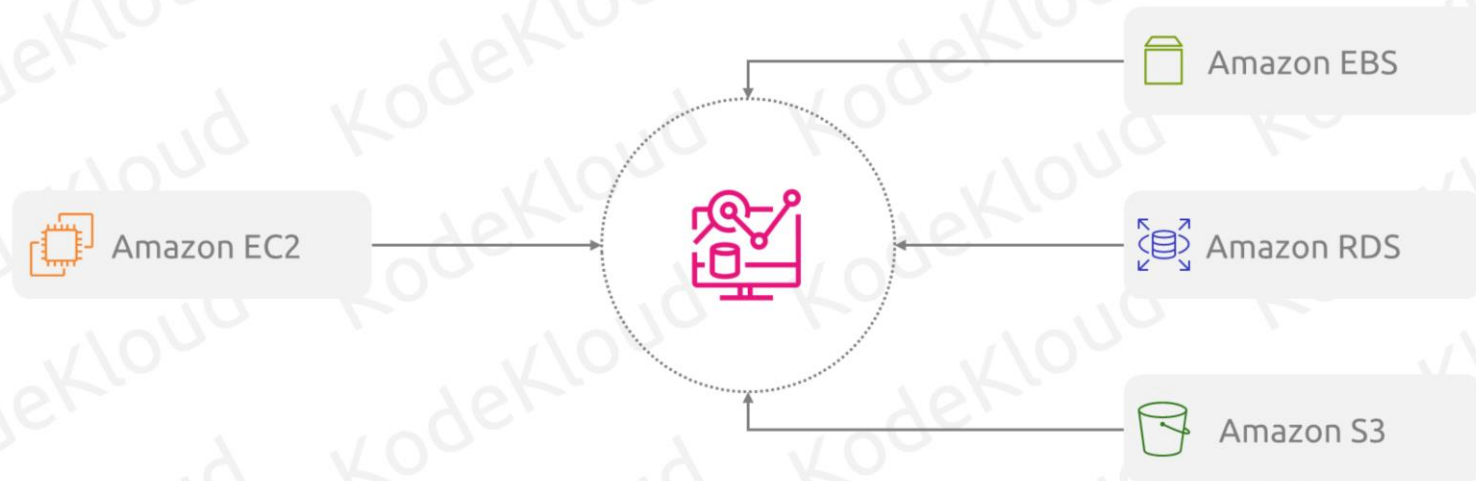
© Copyright KodeKloud

Cloudwatch Dashboard

Amazon CloudWatch dashboards are customizable home pages in the CloudWatch console that you can use to monitor your resources in a single view, even those resources that are spread across different Regions. You can use CloudWatch dashboards to create customized views of the metrics and alarms for your AWS resources.

With dashboards, you can have a common view of critical resource and application measurements that can be shared by team members for faster communication flow during operational events.

CloudWatch Metrics



© Copyright KodeKloud

Cloudwatch Metrics

1. Metrics are data about the performance of your systems.
2. By default, many services provide free metrics for resources (such as Amazon EC2 instances, Amazon EBS volumes, and Amazon RDS DB instances).

3. You can also enable detailed monitoring for some resources, such as your Amazon EC2 instances, or publish your own application metrics.
4. Amazon CloudWatch can load all the metrics in your account (both AWS resource metrics and application metrics that you provide) for search, graphing, and alarms.

Metric data is kept for 15 months, enabling you to view both up-to-the-minute data and historical data.

CloudWatch Agent



© Copyright KodeKloud

•CloudWatch Agent: A Monitoring Solution

What is it?

- Lightweight software for collecting and sending system and application-level metrics and logs from AWS resources to Amazon CloudWatch.

Key Features

- Real-time Monitoring: Collect and visualize performance data.
- Centralized Logging: Consolidate logs for analysis and troubleshooting.
- Custom Metrics: Easily monitor custom application metrics.

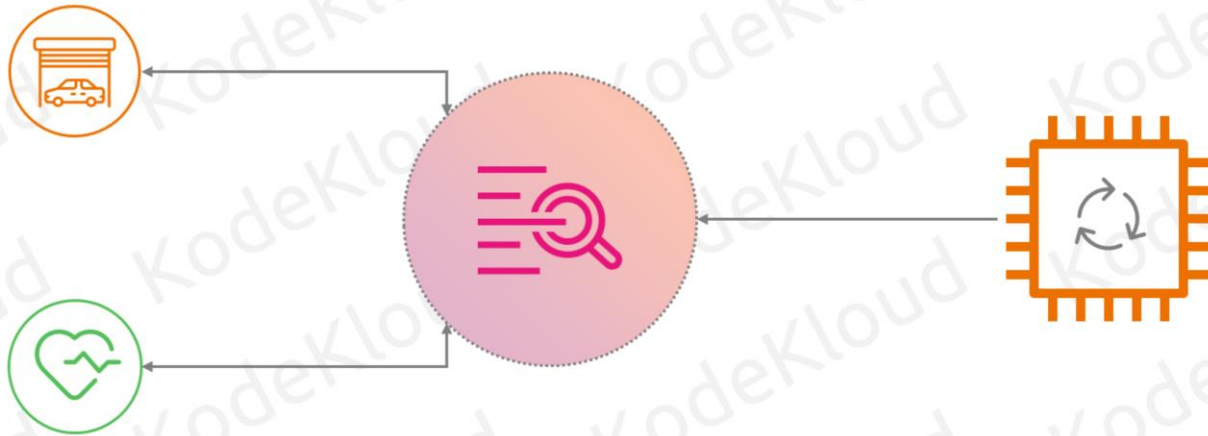
Benefits

- Scalable & Cost-Efficient
- Agent for EC2, on-premises, and hybrid environments.
- Seamless Integration with AWS services

Why CloudWatch Agent?

- Gain Insights
- Ensure Availability
- Optimize Resources

CloudWatch Logs



© Copyright KodeKloud

You can use Amazon CloudWatch Logs to monitor, store, and access your log files from Amazon Elastic Compute Cloud (Amazon EC2) instances, AWS CloudTrail, Route 53, and other sources.

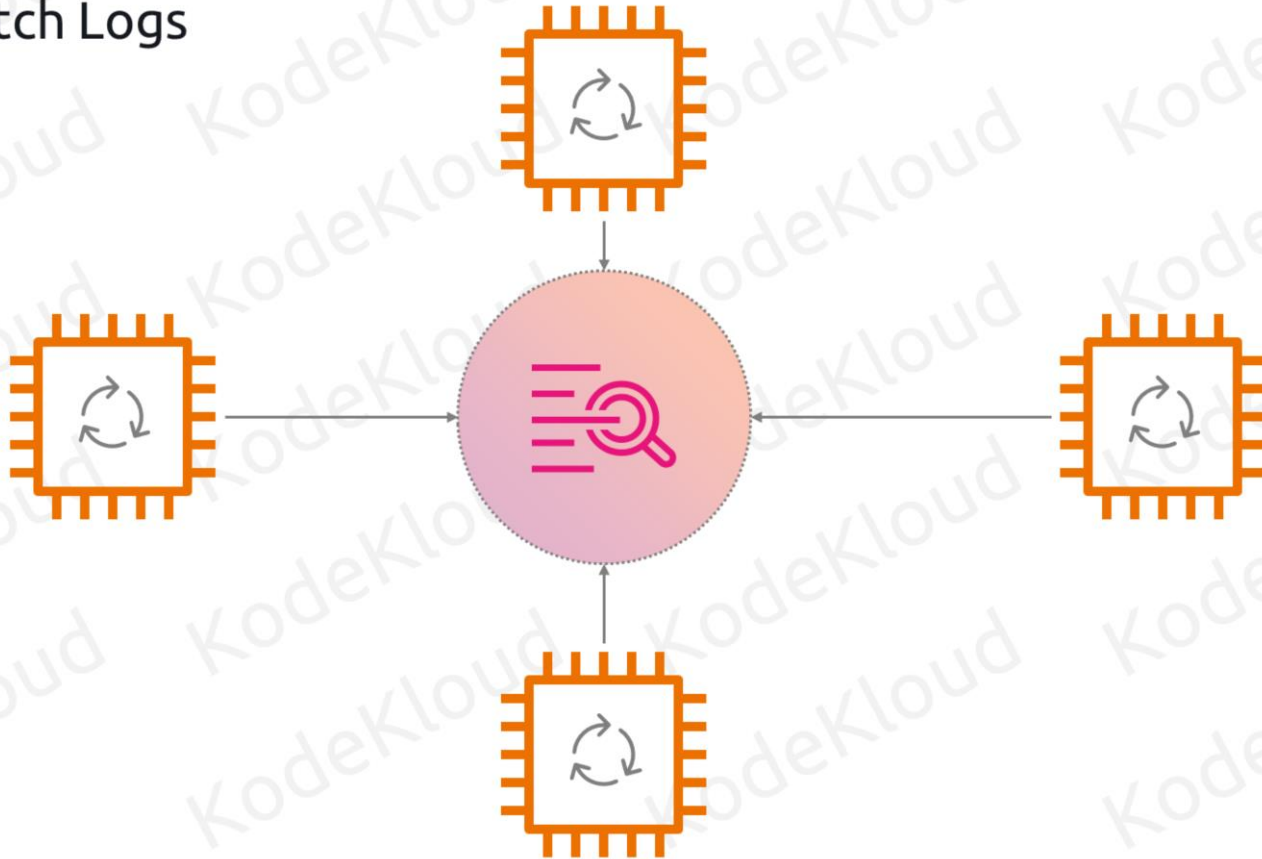
CloudWatch Logs enables you to centralize the logs from all of your systems, applications, and AWS services that you

use, in a single, highly scalable service.

You can then easily view them, search them for specific error codes or patterns, filter them based on specific fields, or archive them securely for future analysis.

CloudWatch Logs enables you to see all of your logs, regardless of their source, as a single and consistent flow of events ordered by time.

CloudWatch Logs



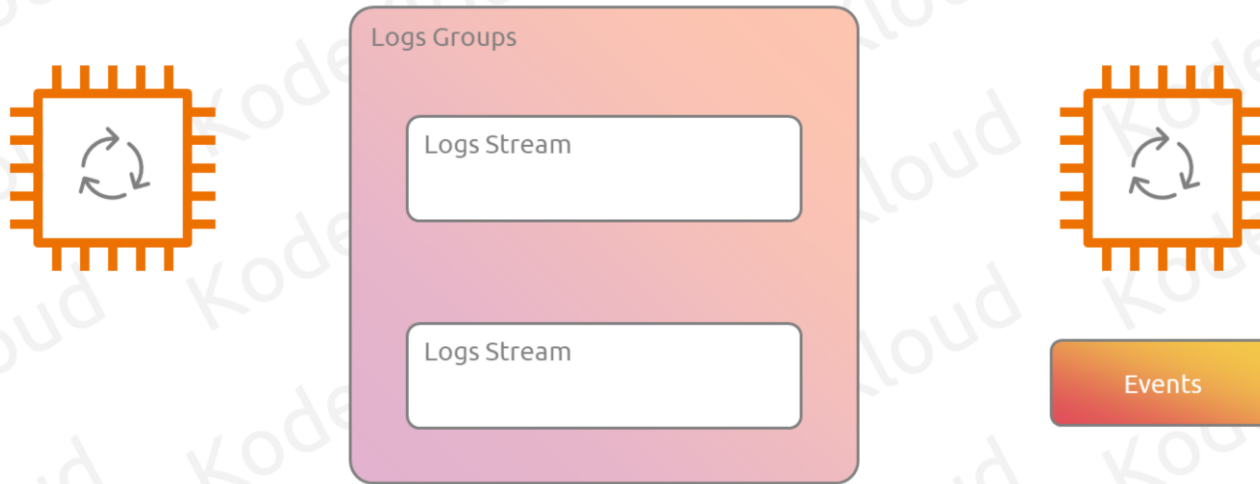
© Copyright KodeKloud

CloudWatch Logs enables you to centralize the logs from all of your systems, applications, and AWS services that you use, in a single, highly scalable service.

You can then easily view them, search them for specific error codes or patterns, filter them based on specific fields, or archive them securely for future analysis.

CloudWatch Logs enables you to see all of your logs, regardless of their source, as a single and consistent flow of events ordered by time.

CloudWatch Logs Concept



© Copyright KodeKloud

Let's understand few internal concepts of cloudwatch logs with a real work example

Event : Lets assume the EC2 instances are running a webside which is hosted on apache or nginx, the access and error logs which are generated by the service are called events, which has timestamp and the raw event data.

Stream: A log stream is a sequence of log events that share the same source,

Logs group : Log groups define groups of log streams that share the same retention, monitoring, and access control settings. Each log stream has to belong to one log group

EC2 Pricing

© Copyright KodeKloud

Hello, welcome to this lecture where we will learn about EC2 pricing.

Cloud Growth



© Copyright KodeKloud

Let's look at cloud journey of a company

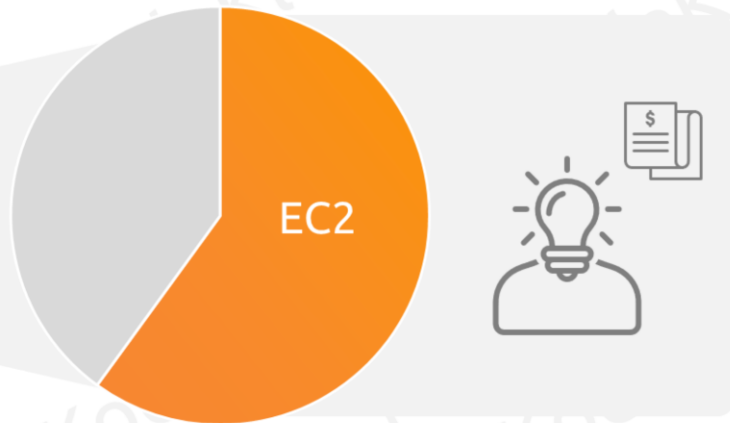
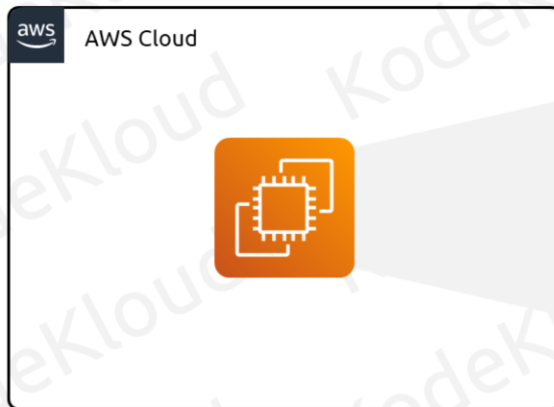
Let's assume you are building a new website, normally a company goes through different phases, like,

- start-up phase
- steady phase
- growth phase.

- Start-up phase :
 - during start-up phase we have limited users or you are trying to find the market fit your product,
 - where you are still experimenting. During this stage cloud resources are created and destroyed on-demand,
 - in this phase it is important to have the flexibility to pay only for cloud usage without any upfront commitment, Agility is key at this stage.
- Steady Phase :
 - Next is steady phase, during this phase the market fit is found and you have regular users 24/7 using your site.
 - During this stage you know the average number of users visiting your site and average cloud usage, hence the growth is predictable.
- Growth Stage :
 - At this stage your cloud usage will vary every hour and it will be different between day and night.
 - as the website grows so does the features and complexities, like running users analysis , batch jobs etc.
 - the website is famous, you have large amount of users accessing your website day and night,

AWS understands this growth pattern and it offers its cloud users with different pricing models to choose based on the requirement and its cloud spending.

EC2 Cost Management

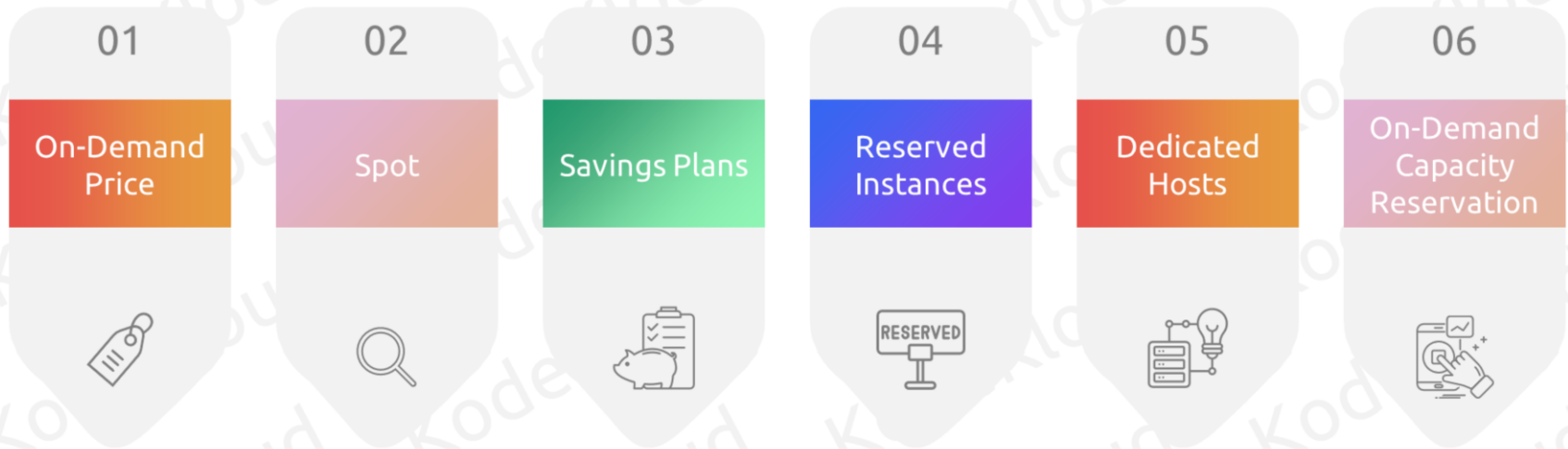


© Copyright KodeKloud

>EC2 instance is one the most commonly used AWS service in many organization.

> And it is also one of the major contributor to AWS billing, as a cloud administrator it is important to understand billing of a service, so that you can make required optimization to bring down your cost.

AWS EC2 Pricing Model



© Copyright KodeKloud

AWS Offers different pricing model for EC2 instance which we can choose based on our need and usage,

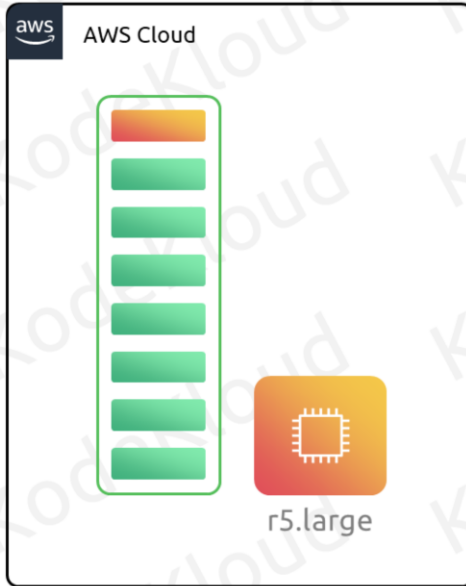
- >1. on-demand price
- >2. Spot
- >3. Savings plans
- >4. Reserved Instances

>5. Dedicated Hosts

>6. on-demand capacity reservation

Understanding how these pricing models function inside is critical for cost-effective consumption.

On-Demand



Provision
r5.large



Features



Use Case



> When AWS launched a new region, server capacity is ready for use.

> When a user request for a EC2 instance of a specific size, AWS will deliver on-demand and start billing the users for every second. This is called on on-demand instance and on-demand price.

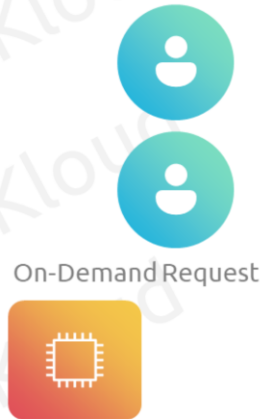
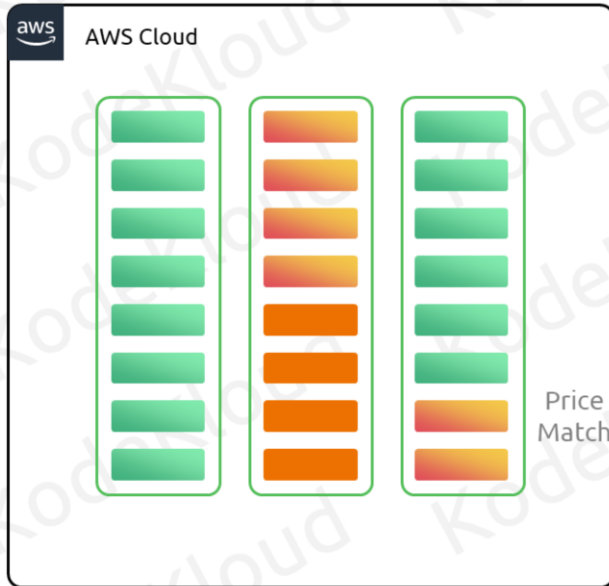
> Features of On-Demand

- * High price
- Flexible with per second billing
- Not commitment, delete anytime and pay only for the usage.

> Use case

- Flexibility and experimentation
- Unplanned workload

Spot Instance



Features



Use Case



Spot Instance

>When AWS launched a new region, large amount server capacity is ready for use, but the usage is not 100% always,

- based on the time of the day the capacity requirements keeps changing,
- for example: we will have instance autoscaled during day time to meet the demand and downsize as the traffic

reduces.

- AWS has spare capacity at any given point in time, Amazon EC2 Spot Instances let you take advantage of unused EC2 capacity in the AWS cloud and are available at a discount of up to 90% compared to On-Demand prices.

The price of a spot instance is decided based on the capacity available and demand for spot instance, user will place their bids for the spot instance and the highest bidder will get the EC2 instance until the price does not change.

> But the catch is, when a request comes in for on-demand instance the spot instances are deleted with a notice of two minutes automatically and users do not have any control over the deletion process.

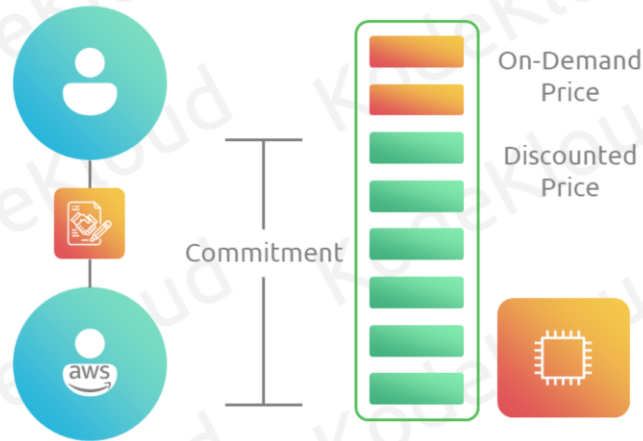
> Features of Spot Instance

- 90% cost effective
- Not commitment
- Price is based on the demand and supply

> Use case

- Fault tolerant or stateless workloads
- Applications that have flexible start and end time , like hadoop cluster, batch processing etc
- Applications that can run on heterogeneous hardware

Saving Plans



Compute Savings Plan



EC2 Instance Savings Plan



© Copyright KodeKloud

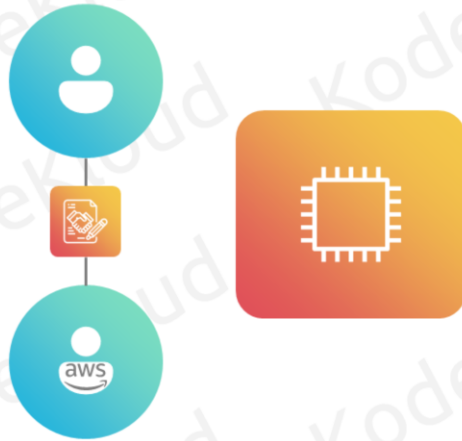
- Savings Plans are a flexible pricing model that offer low prices
- on Amazon EC2,
- in exchange for a commitment to a consistent amount of usage (measured in \$/hour) for a 1 or 3 year term.
- When you sign up for a Savings Plan,
- > > you will be charged the discounted Savings Plans price for your usage up to your commitment.
- For any usage above the commitment, users will be charged on-demand pricing.

- AWS offers two types of Savings Plans, Compute Savings Plans
- Compute Savings Plans provide the most flexibility and help to reduce your costs by up to 66%.
- These plans automatically apply to EC2 instance usage regardless of instance family,
 - size,
 - AZ (availability zone)
 - Region, OS or tenancy.
 - For example, with Compute Savings Plans, you can change from C4 to M5 instances, shift a workload from EU (Ireland) to EU (London), automatically continue to pay the Savings Plans price.

➤ ~~EC2 Instance Savings Plans~~

- EC2 Instance Savings Plans provide the lowest prices,
- offering savings up to 72% in exchange for commitment to usage of individual instance families in a Region (e.g. M5 usage in N. Virginia).
- This automatically reduces your cost on the selected instance family in that region regardless of AZ, size, OS or tenancy.
- EC2 Instance Savings Plans give you the flexibility to change your usage between instances within a family in that region.
 - For example, you can move from c5.xlarge running Windows to c5.2xlarge running Linux and automatically benefit from the Savings Plan prices.

Reserved Instances



Standard Reserved Instances



Convertible Reserved Instances



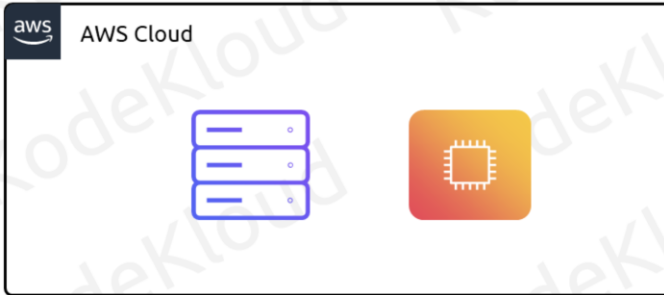
© Copyright KodeKloud

- Reserved Instances are first generation saving plan option,
- provided by AWS,
- it provide you with a significant discount compared to On-Demand Instance pricing, similar to savings plans in exchange for a commitment
- For ec2 instances.

AWS offers two types of Reserved Instances Pricing, Standard and Convertible

- Standard Reserved Instances provide you with a significant discount (up to 72%) compared to, On-Demand Instance pricing, and can be purchased for a 1-year or 3-year term.
- Customers have the flexibility to change the Availability Zone, the instance size, and networking type of their Standard Reserved Instances.
- Convertible Reserved Instances ➤ provide you with a significant discount (up to 66%) compared to On-Demand Instances and can be purchased for a 1-year or 3-year term.
- Purchase Convertible Reserved Instances if you need additional flexibility, such as the ability to use different instance families, operating systems, or tenancies over the Reserved Instance term.

Dedicated Hosts



Instance Family	Sockets	Physical Cores	Medium	Large	xlarge	2xlarge
c7g	3	192	192	96	48	24

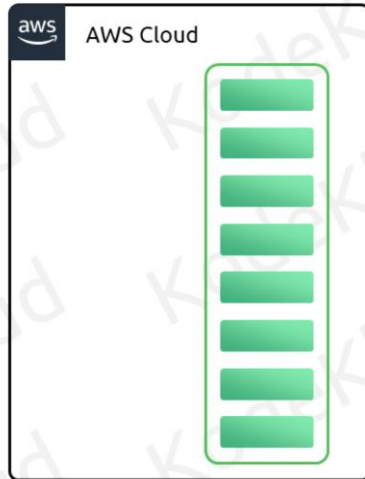
> A Dedicated Host is an offering from AWS, it is a physical server fully dedicated for your use, when you purchase a dedicated host from AWS users have the option to choose number of sockets and cores.

> Let's look at some of the key Features

> This provides you the flexibility to bring your on-premise license modes.

- Workload which needs physical server
 - Maintain control on the server and what runs on it, but offload the maintains to the AWS
 - Price is per dedicate host irrespective of number of EC2 instance that is launched on the dedicate host.
 - Price discount can be availed by using Reserved instance and Saving plan pricing.
-
- The table provides you an example configuration of a dedicate host:
 - A dedicated host with 3 sockets and 19 cores, can accommodate 192 medium , 96 large , 48 xlarge or 23 2xlarge instances.

On-Demand Capacity Reservation



Features



Capacity assurance



Regulatory HA requirement



Disaster recovery



RI and savings plan supported

> On-Demand Capacity Reservations is an offering from AWS which enable you to reserve compute capacity for your EC2 instances in a specific Availability Zone for a duration, and customers are charged irrespective of usage.

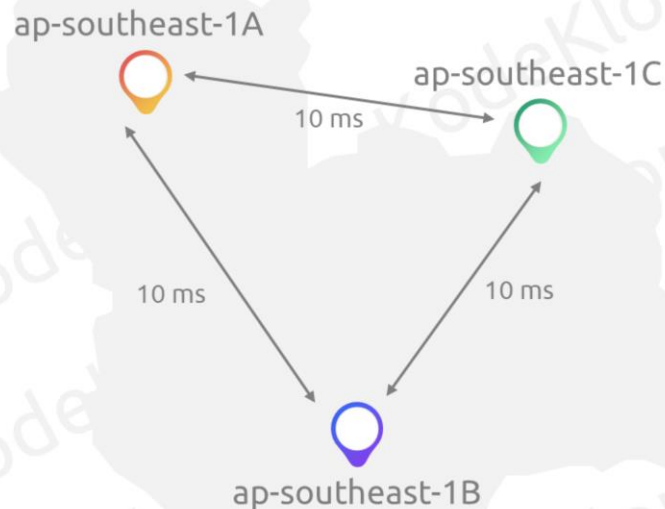
> On-Demand Capacity Reservations are recommended for
> Business-critical events or workloads that require capacity assurance

- Workloads that need to meet regulatory requirements for high availability
- Disaster recovery
- Price discount can be availed by using Reserved instance and Saving plan pricing.



EC2 Placement Groups

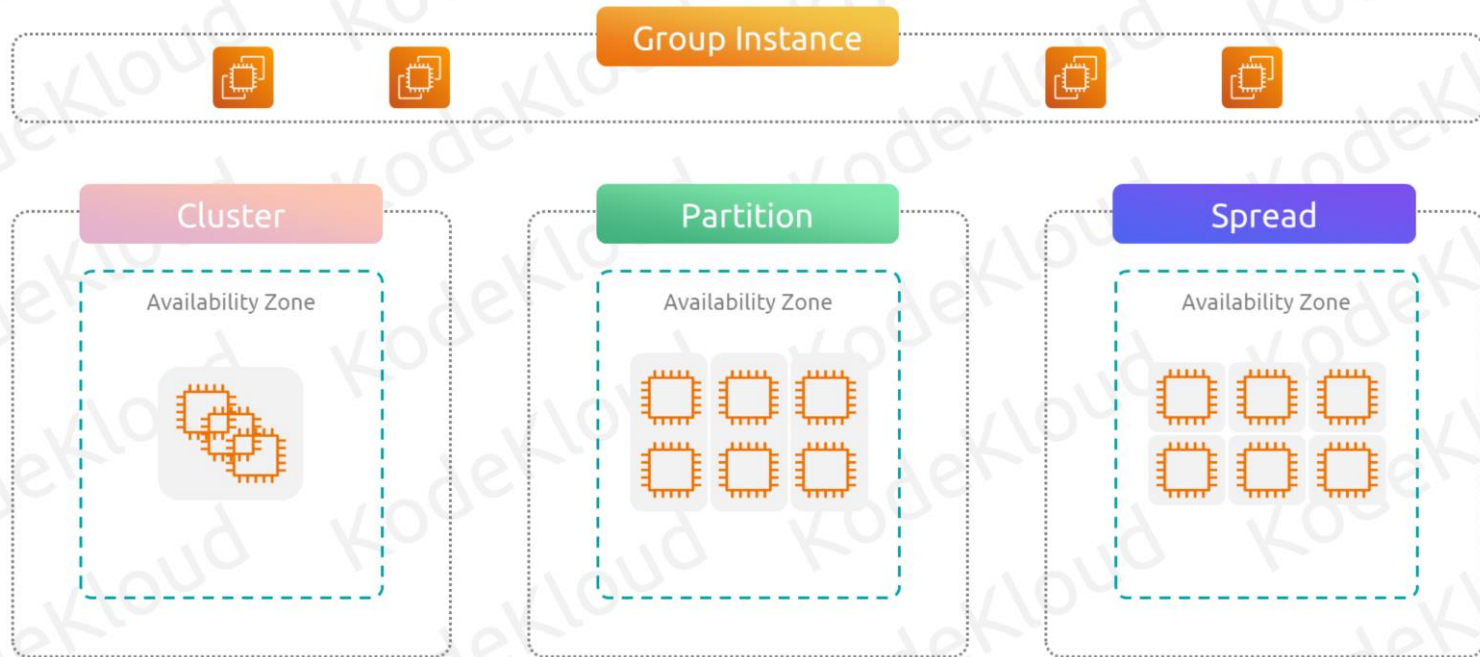
Need for Placement Groups



© Copyright KodeKloud

- Assume you have > deployed your > application in Singapore region and application is spread across multiple availability zone,
- which could have latency up-to 10ms across AZ's .
application like (spark cluster or HPC systems) cannot afford such latency. To over come this limitation AWS introduced placement groups. AWS Placement Group is a networking construct that allows you to control the placement of your EC2 instances to meet specific needs.

Placement Groups



© Copyright KodeKloud

> You can use *placement groups* to influence the placement of a group of *interdependent* instances to meet the needs of your workload. Depending on the type of workload, you can create a placement group using one of the following placement strategies:

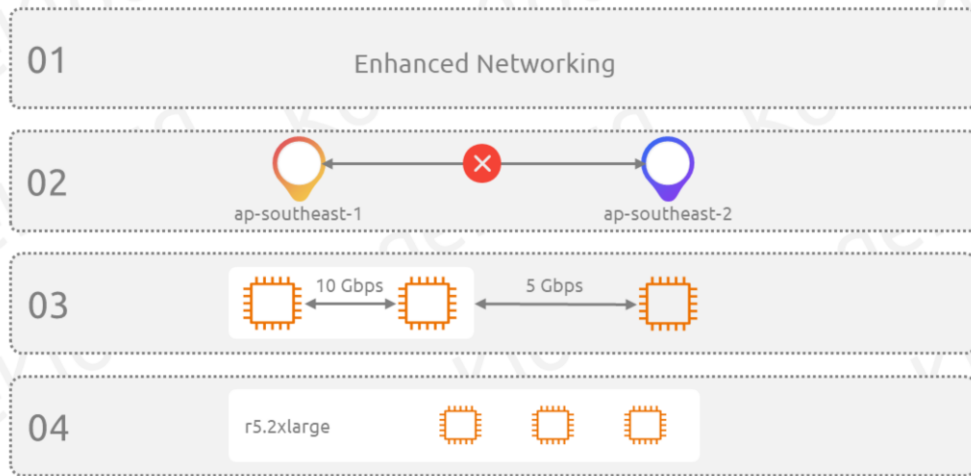
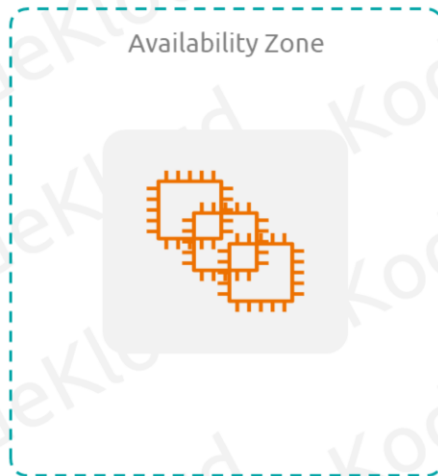
> **Cluster** placement group packs instances close together, inside a specific Availability Zone, Used for low-latency, high-throughput networking, ideal for high performance computing (HPC)

> Partition placement group spreads your instances across logical partitions, such that groups of instances in one partition do not share the underlying hardware with groups of instances in different partitions. Ensures that instances are placed on distinct physical hardware, reducing the risk of simultaneous failures. Useful for applications needing high availability.

> Spread placement group Similar to Spread, but instances are grouped into partitions, each on different sets of hardware. Best for large distributed and replicated workloads, like Hadoop or Cassandra.

Let's look at each type of placement group in detail.

Placement Group – Cluster



© Copyright KodeKloud

> As we already know cluster placement group, packs instances close together inside an Availability Zone.

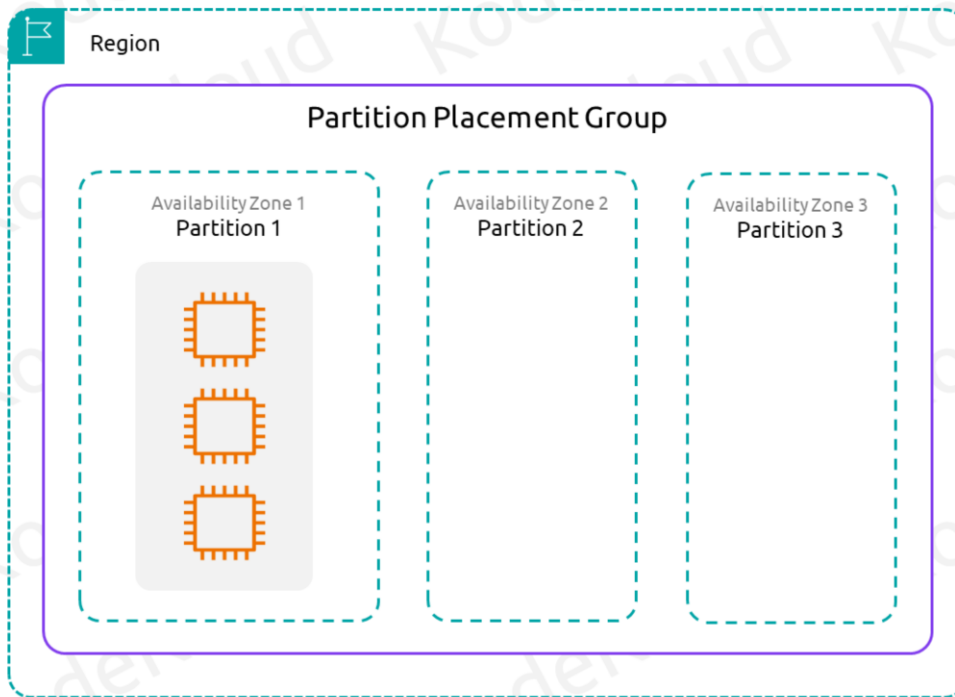
Placement Groups have certain rules and limitations to consider:

- > must use instance type that supports enhanced networking
- A cluster placement group can't span multiple Availability Zones.
- Instances within a cluster placement group can use up to 10 Gbps for single-flow traffic . Instances that are not within

a cluster placement group can use up to 5 Gbps

- It is recommend using the same instance type for all instances in a cluster placement group.

Placement Group – Partition



01 | Overcome hardware failure

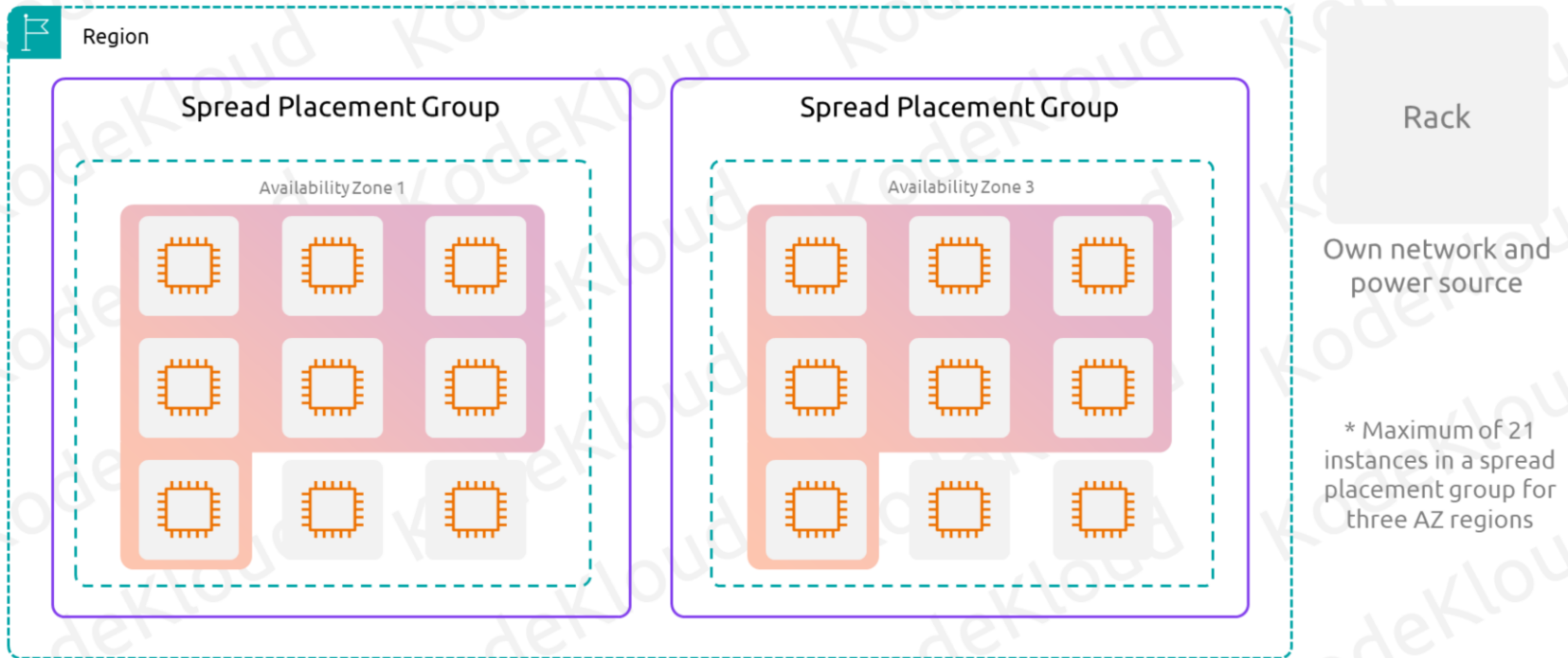
02 | Maximum 7 partitions per AZ

© Copyright KodeKloud

- > Amazon EC2 divides each group of EC2 into logical segments called partitions.
- > Amazon EC2 ensures that each partition within a placement group has its own set of racks and No two partitions within a placement group share the same racks, allowing you to isolate the impact of hardware failure within your application.
- > A partition placement group can have partitions in multiple Availability Zones in the same Region

- Partition placement group is used to over hardware failure
- maximum of seven partitions per Availability Zone, is the only rule when it comes to partition placement group.

Placement Group – Spread



© Copyright KodeKloud

- Rack Spread placement groups can distribute instances across multiple available zones in the same Region
- each rack has its own network and power source and each EC2 instance are placed within a rack.
- For rack spread level placement groups, you can have a maximum of seven running instances per Availability Zone per group.
- It supports maximum of seven running instances per Availability Zone per group.

- In a region with 3 available zone, a rack spread placement group can only have 21 EC2 instances.



KodeKloud

© Copyright KodeKloud

Follow us on <https://kodekloud.com/> to learn more.